

MONOLITHIC ALGEBRAIC MULTIGRID PRECONDITIONERS FOR THE STOKES EQUATIONS*

ALEXEY VORONIN[†], SCOTT MACLACHLAN[‡], LUKE N. OLSON[†], AND
RAYMOND S. TUMINARO[§]

Abstract. We investigate a novel monolithic algebraic multigrid (AMG) preconditioner for the Taylor–Hood ($\mathbb{P}_2/\mathbb{P}_1$) and Scott–Vogelius ($\mathbb{P}_2/\mathbb{P}_1^{disc}$) discretizations of the Stokes equations. The algorithm is based on the use of the lower-order $\mathbb{P}_1$ iso $\mathbb{P}_2/\mathbb{P}_1$ operator within a defect-correction setting, in combination with AMG construction of interpolation operators for velocities and pressures. The preconditioning framework is primarily algebraic, though the $\mathbb{P}_1$ iso $\mathbb{P}_2/\mathbb{P}_1$ operator must be provided. We investigate two relaxation strategies in this setting. Specifically, a novel block factorization approach is devised for Vanka patch systems, which significantly reduces storage requirements and computational overhead, and a Chebyshev adaptation of the LSC-DGS relaxation from [54] is developed to improve parallelism. The preconditioner demonstrates robust performance across a variety of two-dimensional and three-dimensional Stokes problems, often matching or exceeding the effectiveness of an inexact block triangular (or Uzawa) preconditioner, especially in challenging scenarios such as elongated-domain problems.

Key words. Stokes equations, algebraic multigrid, monolithic multigrid, Vanka relaxation, LSC-DGS relaxation, Scott–Vogelius

MSC codes. 65N55, 49M27, 49M20, 76M99

DOI. 10.1137/23M1584514



See reproducibility of
computational results
at end of the article.

1. Introduction. Differential equations and discrete problems with saddle-point structure arise in many scientific and engineering applications. Common examples are the incompressible Stokes and Navier–Stokes equations, where the incompressibility constraint leads to saddle-point problems both at the continuum level and for many common discretizations [16, 6]. Discretizations of these systems using stable

*Submitted to the journal’s Numerical Algorithms for Scientific Computing section July 7, 2023; accepted for publication (in revised form) August 23, 2024; published electronically February 5, 2025.
<https://doi.org/10.1137/23M1584514>

Funding: The work of the second author was partially supported by an NSERC Discovery Grant. The fourth author was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Applied Mathematics program. Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC (NTESS), a wholly owned subsidiary of Honeywell International for the U.S. Department of Energy’s National Nuclear Security Administration (DOE/NNSA) under contract DE-NA0003525. This written work is authored by an employee of NTESS. The employee, not NTESS, owns the right, title, and interest in and to the written work and is responsible for its contents. Any subjective views or opinions that might be expressed in the written work do not necessarily represent the views of the U.S. Government. The publisher acknowledges that the U.S. Government retains a nonexclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this written work or allow others to do so, for U.S. Government purposes. The DOE will provide public access to results of federally sponsored research in accordance with the DOE Public Access Plan.

[†]Department of Computer Science, University of Illinois at Urbana-Champaign, Urbana, IL 61801 USA (voronin2@illinois.edu, <http://alexey-voronin.github.io/>, lukeo@illinois.edu, <http://lukeo.cs.illinois.edu/>).

[‡]Department of Mathematics and Statistics, Memorial University of Newfoundland, St. John’s, NL, Canada (smaclachlan@mun.ca, <https://www.math.mun.ca/smaclachlan/>).

[§]Sandia National Laboratories, Livermore, CA 94551 USA (rstumin@sandia.gov, <https://www.sandia.gov/ccr/staff/raymond-s-tuminaro/>).

finite-element pairs lead to 2×2 block-structured linear systems, where the lower-right diagonal block is a zero-matrix. The indefiniteness in these matrices, along with the coupling between the two unknown types, prevents the straightforward application of standard geometric or algebraic multigrid (AMG) methods. Several successful preconditioners exist for these systems and are typically built on approximate block-factorizations [16, 2, 21] or monolithic geometric multigrid [9, 48, 34, 2, 7, 46]. While some AMG algorithms have been proposed [52, 53, 37, 28, 44], general-purpose AMG methods that are successful for the numerical solution of a wide variety of discretizations of the Stokes equations are recent (and still rare) [39, 40, 4, 3]. In this work, we propose a robust and efficient AMG method for the Stokes $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization and show how the method can be leveraged to develop effective preconditioners for other discretizations, building on the geometric multigrid methods in [50].

Monolithic geometric multigrid methods for these problems are well known and date back to some of the earliest multigrid approaches for finite-difference discretizations [8, 48, 34, 38]. In these works and in later works focused on finite-element discretizations [31, 29, 33, 17, 24, 2, 1], simultaneous geometric coarsening of velocity and pressure fields leads to coarse-grid operators that naturally retain stability on all grids of the hierarchy. This coarsening in combination with monolithic relaxation schemes, such as Vanka [48], Braess and Sarazin [7], or distributive [9, 54] relaxation, has led to effective multigrid convergence for a wide variety of problems and discretizations. Despite the success and effectiveness of these methods, they are not without their challenges, particularly when considering their extension to AMG approaches.

Two difficulties arise when attempting to extend these monolithic methods to AMG approaches. First, it is difficult to construct independent algebraic coarsenings of the velocity and pressure fields that maintain algebraic properties of the fine-grid discretization (such as the relative number of velocity and pressure degrees of freedom (DoFs)) after Galerkin coarsening. As a result, it is possible that a stable discretization on the finest grid yields unstable discretizations on coarser grids in the AMG hierarchy. Second, most inf-sup stable finite-element discretizations of the Stokes equations rely on higher-order finite-element spaces (particularly for the discretized velocity), yielding discretization matrices that are far from the M-matrices for which classical AMG heuristics are known to work best [27, 42]. Consequently, even if stability problems are avoided, convergence of AMG-based solvers can deteriorate as the order of the discretization increases.

Many existing AMG approaches for the Stokes equations attempt to bypass coarsening issues by constraining the coarsening within AMG to loosely mimic properties of geometric multigrid. Specifically, it is possible to roughly maintain the ratio between the number of velocity DoFs and the number of pressure DoFs throughout the multigrid hierarchy by leveraging either geometric physical DoFs locations or element information [52, 53, 28, 37, 44]. While successful, this introduces additional heuristics that depend on both the fine-level discretization and the geometry of the mesh in the AMG process. Change-of-variable transformations have been used successfully to construct monolithic AMG preconditioners for saddle-point-type systems such as the Stokes equations [55, 39, 40, 4, 3]. These transformations convert the system into a similar one with a scalar elliptic operator in the bottom-right block, making it more suitable for classical AMG with unknown-based coarsening and pointwise relaxation. The method has been theoretically analyzed, showing a uniform bound on the spectral radius of the iteration matrix using a single step of damped Jacobi relaxation [39]. However, when applying these methods to Oseen problems using a two-level approach, it has been demonstrated that the use of more effective relaxation approaches, such

as Gauss–Seidel, is necessary to achieve fast convergence [4]. While this approach has been extended to multilevel contexts, as demonstrated in [3], ensuring robust and cost-effective convergence requires two pieces: first, the efficient implementation of relaxation techniques like Gauss–Seidel and successive over-relaxation, which are difficult to parallelize, and second, effective algorithms for sparsification of coarse-grid matrices.

In this paper, we propose a new AMG framework for the solution of the discretized Stokes equations, based on preconditioning higher-order discretizations by smoothed aggregation (SA) AMG applied to the low-order $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization. We employ both Vanka-style relaxation and a generalization of the least-squares commutator distributive Gauss–Seidel (LSC-DGS) method from [54]. These are applied to the original high-order discretization and each level within the AMG hierarchy, allowing us to compare their relative performances. The use of a low-order preconditioner (in a classical multigrid *defect-correction* (DC) approach) allows us to avoid issues that arise when applying AMG directly to higher-order discretizations, leading to robust convergence with low computational complexities. In [50], we considered a similar strategy within geometric multigrid for the $\mathbf{Q}_2/\mathbf{Q}_1$ discretization, using DC based on the $\mathbf{Q}_1\text{iso}\mathbf{Q}_2/\mathbf{Q}_1$ discretization. Here, we extend that work to both structured and unstructured triangular and tetrahedral meshes, using both Taylor–Hood (TH), $\mathbf{P}_2/\mathbb{P}_1$, and Scott–Vogelius (SV), $\mathbf{P}_2/\mathbb{P}_1^{disc}$, discretizations on the finest grids. To achieve scalable results, particularly on unstructured grids, we examine best practices for AMG on the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization, including the preservation of stability on coarse levels of the hierarchy. We note that this also provides insight into achieving scalable results applying AMG directly to the $\mathbf{P}_2/\mathbb{P}_1$ discretization, and we compare this to our DC approach.

The remainder of this paper is structured as follows. In section 2, we introduce the Stokes equations and the accompanying discretizations. Section 3 describes the monolithic AMG framework and the construction of the low-order preconditioner for high-order discretizations of the Stokes equations. Section 4 presents numerical results, demonstrating the robustness of the preconditioner for a set of test problems, as well as a comparison of the computational cost between the various AMG-based preconditioners. In section 5, we present numerical results obtained by applying the most effective preconditioners identified in the previous section to more complex application problems. Section 6 provides concluding remarks, with future research directions outlined in section 7.

2. Discretization of the Stokes equations. In this paper, we consider the steady-state Stokes problem on a bounded Lipschitz domain $\Omega \subset \mathbb{R}^d$, for $d = 2$ or 3 , with boundary $\partial\Omega = \Gamma_N \cup \Gamma_D$, given by

$$\begin{aligned} (2.1a) \quad & -\nabla^2 \mathbf{u} + \nabla p = \mathbf{f} && \text{in } \Omega, \\ (2.1b) \quad & -\nabla \cdot \mathbf{u} = 0 && \text{in } \Omega, \\ (2.1c) \quad & \mathbf{u} = \mathbf{g}_D && \text{on } \Gamma_D, \\ (2.1d) \quad & \frac{\partial \mathbf{u}}{\partial \mathbf{n}} - \mathbf{n}p = \mathbf{g}_N && \text{on } \Gamma_N. \end{aligned}$$

Here, $\mathbf{u}$ is the velocity of a viscous fluid, p is the pressure, $\mathbf{f}$ is a forcing term, $\mathbf{n}$ is the outward pointing normal to $\partial\Omega$, $\mathbf{g}_D$ and $\mathbf{g}_N$ are given boundary data, and Γ_D and Γ_N are disjoint Dirichlet and Neumann segments of the boundary. In a variational formulation, the natural function space for the velocity is $\mathcal{H}_{\mathbf{g}_D}^1(\Omega)$, the subset of $\mathcal{H}^1(\Omega)$ constrained to match the essential boundary condition (BC) in (2.1c). If the

velocity is specified everywhere along the boundary, which requires $\int_{\partial\Omega} \mathbf{g}_D \cdot \mathbf{n} = 0$, then a suitable function space for the pressure is the space of zero-mean functions in $L^2(\Omega)$ —i.e., $\mathcal{Q} = L^2_0(\Omega)$; this avoids a pressure solution that is unique only up to a constant. In the case where $|\Gamma_N| > 0$, this additional constraint is not needed, and the pressure is assumed to be in $\mathcal{Q} = L^2(\Omega)$ [16].

For both TH and SV element pairs, define finite-dimensional spaces $(\mathbf{V}_h^D, \mathcal{Q}_h) \subset (\mathcal{H}_{\mathbf{g}_D}^1(\Omega), \mathcal{Q})$, where $\mathbf{V}_h^D$ strongly satisfies the Dirichlet BCs on Γ_D specified by (2.1c) (at least up to interpolation error). The resulting weak formulation of (2.1) is to find $\mathbf{u} \in \mathbf{V}_h^D$ and $p \in \mathcal{Q}_h$ such that

$$(2.2a) \quad a(\mathbf{u}, \mathbf{v}) + b(p, \mathbf{v}) = F(\mathbf{v}),$$

$$(2.2b) \quad b(q, \mathbf{u}) = 0$$

for all $\mathbf{v} \in \mathbf{V}_h^0$ and $q \in \mathcal{Q}_h$, where $\mathbf{V}_h^0$ is the same finite-element space as $\mathbf{V}_h^D$, but with zero Dirichlet BCs on Γ_D . Here, $a(\cdot, \cdot)$ and $b(\cdot, \cdot)$ are bilinear forms, and $F(\cdot)$ is a linear form given by

$$a(\mathbf{u}, \mathbf{v}) = \int_{\Omega} \nabla \mathbf{u} : \nabla \mathbf{v}, \quad b(p, \mathbf{v}) = - \int_{\Omega} p \nabla \cdot \mathbf{v}, \quad \text{and} \quad F(\mathbf{v}) = \int_{\Omega} \mathbf{f} \cdot \mathbf{v} + \int_{\Gamma_N} \mathbf{g}_N \cdot \mathbf{v}.$$

From this point forward, we overload the notation and use $\mathbf{u}$ and p to denote the discrete velocity and pressure unknowns in a finite-element discretization of (2.2). Given an inf-sup stable choice of finite-dimensional spaces $(\mathbf{V}_h^D, \mathcal{Q}_h)$, we obtain a saddle-point system of the form

$$(2.3) \quad Kx = \begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ 0 \end{bmatrix} = b,$$

where matrix A corresponds to the discrete vector Laplacian on $\mathbf{V}_h^D$, and B represents the negative of the discrete divergence operator mapping $\mathbf{V}_h^D$ into $\mathcal{Q}_h$. In two dimensions, the dimensions of A and B are given by $A \in \mathbb{R}^{(n_x+n_y) \times (n_x+n_y)}$ and $B \in \mathbb{R}^{n_p \times (n_x+n_y)}$, where n_x and n_y represent the number of velocity nodes for each component of the vector $\mathbf{u}$ (which are equal for the BCs considered here) and n_p represents the number of pressure nodes.

2.1. Finite element discretizations and meshes. For the remainder of the paper, we focus on three different but related discretizations to define finite-dimensional spaces $(\mathbf{V}_h^D, \mathcal{Q}_h)$ on meshes of d -dimensional simplices. The first is the standard TH discretization, $\mathbb{P}_k/\mathbb{P}_{k-1}$, with continuous piecewise polynomials of degree k for the velocity space $\mathbf{V}_h^D$ and continuous piecewise polynomials of degree $k-1$ for the pressure space $\mathcal{Q}_h$. The TH element pair is inf-sup stable for $k \geq 2$ on any triangular (tetrahedral) mesh Ω_h of domain Ω . Here, we focus on the case $k = 2$, leading to the lowest-order pair of TH elements, corresponding to piecewise quadratic velocity components and piecewise-linear elements for the pressure.

The second discretization that we consider is the SV discretization, $\mathbb{P}_k/\mathbb{P}_{k-1}^{disc}$, composed of continuous piecewise polynomials of degree k for the velocity and discontinuous piecewise polynomials of degree $k-1$ for the pressure. The inf-sup stability of the SV discretization depends on the polynomial degree (k) of the function spaces and properties of the mesh used to assemble the system. In [45, 56], the $\mathbb{P}_k/\mathbb{P}_{k-1}^{disc}$ discretization is shown to be stable for $k \geq d$ on any mesh that results from

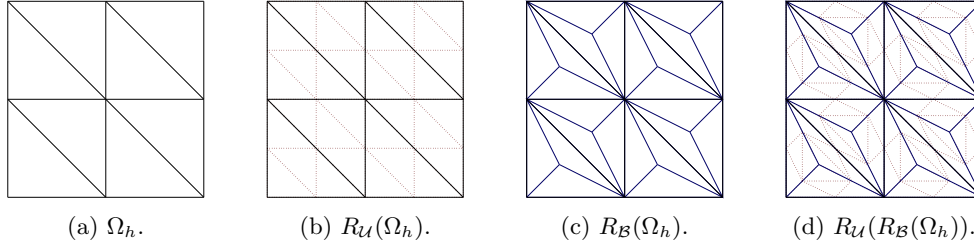


FIG. 1. Mesh refinements. (a) structured 2D triangular mesh; (b) the quadrisection refinement of (a); (c) barycentric mesh (Alfeld split), obtained via barycentric refinement of (a); and (d) quadrisection refinement of (c).

a single step of barycentric refinement of any triangular (tetrahedral) mesh. Such a refinement is depicted in Figure 1(c), where $R_B(\Omega_h)$ denotes the barycentric refinement of mesh Ω_h . In [57, 58], the polynomial order required for inf-sup stability of the SV discretization is lowered to $k \geq d - 1$ when the mesh is given by a Powell–Sabin split of a triangular or tetrahedral mesh, Ω_h , but we do not consider such meshes. An important advantage of the SV discretization over the TH discretization is that the choice of the discontinuous discrete pressure space satisfies the relationship that the divergence of any velocity in $\mathbf{P}_k$ is exactly represented in the pressure space, $\mathbb{P}_{k-1}^{disc}$; therefore, enforcing the divergence constraint weakly leads to velocities that are pointwise divergence-free as well. This is not the case for TH elements, although an additional grad-div stabilization term can be used to achieve stronger pointwise mass conservation [30, 12, 35]. This work focuses on the $\mathbf{P}_2/\mathbb{P}_1^{disc}$ discretization on barycentrically refined meshes in two dimensions.

The third discretization is the low-order $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization. It is well known that a mixed discretization using piecewise-linear finite-elements for both velocities and pressures, such as $\mathbf{P}_1/\mathbb{P}_1$, does not satisfy the necessary inf-sup stability criterion for well-posedness. The $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ gets around this instability by considering first-order discretizations of both velocities and pressures, but on different meshes. Specifically, the $\mathbf{P}_1$ velocity space is discretized on a quadrisection refinement, $R_U(\Omega_h)$, of the mesh, Ω_h , used for the $\mathbb{P}_1$ discretization of the pressure. The $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization is inf-sup stable [18] but is not commonly used due to its relatively low accuracy and computational efficiency. Here, we make use of this discretization not to directly approximate solutions to the Stokes equations, but as an auxiliary operator with which we build preconditioners for the discretizations of interest.

Figure 1 illustrates meshes used in the construction of the $\mathbf{P}_2/\mathbb{P}_1$, $\mathbf{P}_2/\mathbb{P}_1^{disc}$, and $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ systems. In all cases, we consider a given mesh, Ω_h , as shown in Figure 1(a). We denote the quadrisection refinement of Ω_h , by $R_U(\Omega_h)$, as shown in Figure 1(b). Similarly, Figure 1(c) denotes the barycentric refinement of Ω_h , $R_B(\Omega_h)$. For the SV discretization on mesh $R_B(\Omega_h)$, we will make use of a quadrisection refinement of $R_B(\Omega_h)$, denoted $R_U(R_B(\Omega_h))$, shown in Figure 1(d).

3. Monolithic multigrid. In this section, we introduce our multilevel algorithm for solving the Stokes system, $Kx = b$, in (2.3). The algorithm comprises several key components that we detail in subsequent sections, namely,

- the use of the low-order $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization to precondition the system;
- the construction of AMG hierarchies for separate portions of the system,

including the vector Laplacian on the velocity variables (represented by A) and the *div-grad*-like system for pressure;

- a careful algebraic coarsening of the $(\mathbf{u}, p)$ DoFs to mimic behavior observed in efficient geometric multigrid schemes for Stokes;
- an algebraic form of Vanka relaxation for saddle-point systems, with a tuned relaxation parameter;
- a block LU factorization technique for the solution of Vanka patches that greatly reduces storage and computational cost; and
- a variant of the LSC-DGS relaxation as an alternative to Vanka relaxation.

In total, the algorithm derived in this section (and presented in detail in Algorithm 3.1) is algebraic in fashion, requiring only a description of the original (fine) DoFs and a low-order interpolation operator based on the finite-element basis (on the fine level).

3.1. Low-order preconditioning. In [50], we demonstrate that the low-order $\mathbf{Q}_1\text{iso}\mathbf{Q}_2/\mathbf{Q}_1$ finite-element discretization can be leveraged to construct efficient and robust geometric multigrid preconditioners for the higher-order $\mathbf{Q}_2/\mathbf{Q}_1$ discretization of the Stokes equations. In that work, a multilevel DC approach was proposed that we follow here. There are two aims in the present work. First, to extend the approach of [50] from two-dimensional (2D) quadrilateral meshes to triangular and tetrahedral meshes, and to consider more discretizations. Second, we aim to construct the DC preconditioner in an *algebraic* setting without relying on geometric information about the mesh to construct interpolation (or other) operators. While the method is described as an AMG solver, we emphasize that the procedure is best used as a preconditioner within a Krylov method.

In the algorithms that follow, K_0 and K_1 are the Stokes operators assembled using a higher-order discretization (either $\mathbf{P}_2/\mathbf{P}_1$ or $\mathbf{P}_2/\mathbb{P}_1^{disc}$) and the lower-order $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbf{P}_1$ discretization, respectively, where the pressure for the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbf{P}_1$ discretization is defined on the same grid as the higher-order discretization, and the $\mathbf{P}_1\text{iso}\mathbf{P}_2$ velocity is defined on the quadrissection refinement of this mesh. As the spaces associated with K_0 and K_1 are different, transfer operators are needed to map from the low-order mesh to the high-order mesh, denoted by P_0 , and vice versa, denoted by R_0 . Further, let G_1 denote the error-propagation operator for a stationary iterative solver for K_1 . Extending [50], we consider a DC algorithm defined by the two-level iteration scheme with the error-propagation operator

$$(3.1) \quad G_{dc} = (I - \omega_0 M_0^{-1} K_0)^{\nu_2} (I - P_0 (I - G_1^\gamma) K_1^{-1} \eta R_0 K_0) (I - \omega_0 M_0^{-1} K_0)^{\nu_1}.$$

Here, ν_1 relaxation sweeps are first applied to the K_0 operator where the specific relaxation scheme is defined by the choice of M_0^{-1} . This is followed by a coarse/low-order correction given by the middle expression, followed by postrelaxation on K_0 . The low-order correction includes first a high-order residual calculation (the K_0 term), which is restricted using ηR_0 . Then, γ iterations of the G_1 solver are used to produce a low-order approximate solution that is interpolated via P_0 back to the fine space. We note that if $\gamma = \infty$ and G_1 is a convergent iteration, then the middle term reduces to $(I - P_0 K_1^{-1} \eta R_0 K_0)$, which corresponds to solving the coarse problem exactly. As we demonstrate in section 4, an important (and nonstandard) feature of this cycle is the residual weighting operator, η . Here, η is a 2×2 block-diagonal matrix that imposes separate scaling of residual equations corresponding to the momentum, (2.1a), and the continuity, (2.1b), equations, by scalar weights η_u and η_p , and is represented by the following operator:

$$(3.2) \quad \eta = \begin{bmatrix} \eta_u I & \\ & \eta_p I \end{bmatrix}.$$

Depending on the K_0 discretization, we find two useful regimes for the scaling parameters, η_u and η_p . When K_0 comes from the $\mathbb{P}_2/\mathbb{P}_1$ discretization, it is productive to use $\eta_p \approx \eta_u \leq 1$, damping the DC from the K_1 discretization. In contrast, when K_0 comes from the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretization, we find $\eta_p \gg \eta_u \approx 1$, or continuity equation residual overweighting, is the most effective. We note that residual overweighting has been used to accelerate the convergence of non-Galerkin DC multigrid methods by minimizing the discrepancy between levels in other contexts [10, 13]. We also note that, in the work below, we do not introduce additional residual weighting between levels in the multigrid cycle for K_1 , as preliminary experiments found that this did not lead to further improvements in convergence.

We construct the grid transfer operator, P_0 , between the higher-order (K_0) and the low-order (K_1) discretization as a block-diagonal matrix

$$(3.3) \quad P_0 = \begin{bmatrix} P_0^u & \\ & P_0^p \end{bmatrix},$$

where P_0^u and P_0^p correspond to velocity and pressure field interpolation operators. For the K_0 and K_1 system discretized with $\mathbb{P}_2/\mathbb{P}_1$ and $\mathbb{P}_1 \text{iso} \mathbb{P}_2/\mathbb{P}_1$ elements, respectively, the pressure and velocity unknowns are co-located; therefore, it is plausible to use injection, $P_0^u = I$ and $P_0^p = I$, as in [50]. Formally, it is natural to define restriction using element projection (described shortly) to map between two different spaces. However, given the co-located nature of the DoFs, it is convenient to simply use injection to define restriction for $\mathbb{P}_2/\mathbb{P}_1$ and $\mathbb{P}_1 \text{iso} \mathbb{P}_2/\mathbb{P}_1$ discretizations, and so $R_0 = P_0^T$. For the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ operator, K_0 , it is still convenient to use injection for the velocity DoFs, as the $\mathbb{P}_1 \text{iso} \mathbb{P}_2$ velocity DoFs in K_1 are again co-located with those of K_0 , again leading to $P_0^u = R_0^u = I$. For pressure DoFs, however, more complicated mappings are needed, as K_0 uses discontinuous piecewise-linear basis functions, while K_1 uses continuous piecewise-linear basis functions on the same mesh. In two dimensions, there are approximately twice as many elements in a regular triangular mesh as there are nodes, and three pressure DoFs per element for the $\mathbb{P}_1^{disc}$ discretization versus one per node for $\mathbb{P}_1$. Thus, there are approximately six times as many DoFs in the $\mathbb{P}_1^{disc}$ pressure field than in the $\mathbb{P}_1$ pressure field. Since $\mathbb{P}_1 \subset \mathbb{P}_1^{disc}$, P_0^p is naturally given by finite-element interpolation, determining the coefficients in the $\mathbb{P}_1^{disc}$ basis that correspond to a given function in $\mathbb{P}_1$. In the case when discontinuous pressure DoFs are co-located at mesh nodes (with one DoF for each adjacent element), this mapping is simply the duplication of a continuous pressure value at the node to each discontinuous DoF associated with the node, and its discrete structure is easy to infer from the mesh and associated operators. Since $\mathbb{P}_1^{disc} \not\subset \mathbb{P}_1$, however, direct injection or averaging when restricting from $\mathbb{P}_1^{disc}$ to $\mathbb{P}_1$ space is not as trivial. Here, we define R_0^p applied to a function, $p_{dg} \in \mathbb{P}_1^{disc}$, as yielding its corresponding continuous analogue, p_{cg} , by finite-element projection. That is, we compute $p_{cg} \in \mathbb{P}_1$ that satisfies $\langle p_{cg}, q_{cg} \rangle = \langle p_{dg}, q_{cg} \rangle$ for all q_{cg} in the $\mathbb{P}_1$ space. As p_{cg} and q_{cg} can be expressed in terms of basis functions, this leads to solving a matrix equation involving a standard $\mathbb{P}_1$ mass matrix, where the right-hand side is defined in terms of basis functions of the two spaces. In either case, P_0 and R_0 are assembled when K_0 and K_1 are assembled, as the finite-element assembly machinery for the Stokes operators can be repurposed

for the grid transfers. If required, FGMRES preconditioned with a standard scalar AMG solver is used for the mass matrix solve, requiring a reduction of the ℓ_2 norm of the relative residual to 10^{-12} . This is typically achieved without an increase in iteration counts as the mesh is refined. While simpler approximations of the mass matrix inverse are certainly possible, preliminary experiments showed that these result in increased iteration counts for the overall solver. Similar uses of AMG for mass matrix solves have appeared in [22], which reports near-constant iteration counts as the mesh is refined, as well as in [1] and [47].

Remark 3.1 ($\mathbb{P}_2/\mathbb{P}_1$ to $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ grid-transfer operators). A natural question is whether finite-element projection should be used to transfer residuals and corrections between the nonnested velocity spaces $\mathbb{P}_2$ and $\mathbb{P}_1\text{iso}\mathbb{P}_2$. To examine this in a simpler setting, we apply AMG to the $\mathbb{Q}_1\text{iso}\mathbb{Q}_2$ discretization as a preconditioner for the $\mathbb{Q}_2$ discretization of a scalar Poisson problem on the unit square. Defining R_0 via finite-element projection does not significantly improve performance, leading to a maximum gain of only one or two iterations in any given solve. Given the increased cost and complexity of this projection, we do not adopt this for grid transfers between $\mathbb{P}_2$ and $\mathbb{P}_1\text{iso}\mathbb{P}_2$. However, projections may likely show more benefit in other settings, e.g., for even higher-order discretizations.

3.2. Smoothed aggregation AMG for the Stokes operator. The standard two-level error-propagation operator for a multigrid cycle is given by

$$(3.4) \quad G_\ell = (I - \omega_\ell M_\ell^{-1} K_\ell)^{\nu_2} (I - P_\ell K_{\ell+1}^{-1} R_\ell K_\ell) (I - \omega_\ell M_\ell^{-1} K_\ell)^{\nu_1} \text{ for } \ell > 0,$$

where ℓ denotes the level within the multigrid hierarchy, K_ℓ is the level- ℓ system matrix, M_ℓ represents the relaxation operator applied to K_ℓ with damping parameter ω_ℓ , P_ℓ is the interpolation between levels $\ell + 1$ and ℓ , and R_ℓ is the restriction from level ℓ to level $\ell + 1$. G_1 , for example, defines a two-level solver within (3.1) to approximately solve the K_1 system. For AMG methods, $K_{\ell+1}$ is almost always taken as $K_{\ell+1} = P_\ell^T K_\ell P_\ell$, and a standard multigrid algorithm replaces the $K_{\ell+1}$ solve with a recursive application of the two-grid solver. The multilevel cycle is fully specified once the $P_\ell, R_\ell, K_{\ell+1}$, and M_ℓ are defined for all hierarchy levels. In this paper, we take $R_\ell = P_\ell^T$ and $K_{\ell+1} = P_\ell^T K_\ell P_\ell$ within the AMG hierarchy (for $\ell \geq 1$) and, so, only algorithms for P_ℓ and M_ℓ need to be determined.

While popular AMG preconditioners can be directly applied to our $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ system, we find that these generally do not lead to fast convergence. Of course, most popular AMG algorithms were originally designed for scalar PDEs coming from elliptic equations that often give rise to positive-definite matrices. While many AMG techniques are effective on elliptic PDE systems, they are not directly applicable to saddle-point systems. For example, standard SA AMG assumes that damped Jacobi relaxation converges and smooths errors when applied to the system matrix, neither of which is true for saddle-point systems such as K_1 , for which Jacobi is not even well-defined due to the zero diagonal block in the matrix.

To apply AMG to our system, we define block-structured grid transfer operators, as were used in (3.3), and then apply AMG algorithms to define the individual blocks of the interpolation matrix. This structure ensures that fine-level pressures (velocities) interpolate only from coarse-level pressures (velocities), and not from coarse-level velocities (pressures). For the velocity block, since the vector Laplacian is an elliptic PDE discretized using vector $\mathbb{P}_1$ elements, most AMG algorithms can be reliably applied to A to produce a suitable P_ℓ^u for the velocity interpolation operator. AMG

cannot, however, be directly applied to the zero diagonal block associated with pressure, so some auxiliary operator is needed to determine the pressure interpolation. One possible choice for this auxiliary matrix is the associated pressure Laplacian, A_p , discretized using the $\mathbb{P}_1$ pressure basis functions. Using A_p has the disadvantage that it requires an additional matrix for the AMG setup process, albeit one that is easily generated using any finite-element package. An alternative and purely algebraic auxiliary operator is BB^T (or some approximation to $BM_u^{-1}B^T$, where M_u is the velocity mass matrix), perhaps in conjunction with some type of stencil truncation [44]. BB^T represents a discrete Laplace operator, although it generally has a somewhat wider stencil than A_p , which can prove challenging for AMG coarsening algorithms. In the remainder of the paper, we use A_p , assembled on the fine grid, to avoid the introduction of additional tuning parameters associated with truncating BB^T (and in approximating the inverse of M_u to form an approximation to $BM_u^{-1}B^T$).

Algorithm 3.1 details our multigrid hierarchy setup for the 2D case, assuming either direct access to each block of the Stokes systems in (2.3) or being able to infer A and B from K_1 , based on the knowledge of the total number of discrete DoFs in each field. While a systems version of SA-AMG could be applied directly to the vector Laplacian, we instead apply a scalar version of SA-AMG to each component of the vector Laplacian. As expected, the two approaches yield similar aggregates and interpolation operators, assuming the systems SA-AMG algorithm is provided the 2D near-kernel space,

$$(3.5) \quad \mathcal{N}_v = \begin{bmatrix} \mathbf{1}_{n_x} & \\ & \mathbf{1}_{n_y} \end{bmatrix},$$

where $\mathbf{1}_k$ refers to a length k vector with all entries equal to one. We note that the systems approach may be advantageous when considering problems with complex BCs or generalized operators where A is not a block-diagonal matrix.

Algorithm 3.1 SA-AMG hierarchy setup for 2D Stokes system.

```

1: Input:  $K_1 = \begin{bmatrix} A & B^T \\ B & \end{bmatrix}$ , Stokes system where  $B \in \mathbb{R}^{n_p \times (n_x + n_y)}$ 
2:    $A_p$ , pressure stiffness matrix or a truncated version of  $BB^T$ 
3: Output:  $K_1, \dots, K_{\ell_{max}}$ , Grid hierarchy
4:    $P_1, \dots, P_{\ell_{max}-1}$ , Interpolation Operators
5:
6:  $A_x, A_y = \text{split}(A)$  ▷ Split vector Laplacian componentwise
7:  $\{P_1^x, \dots, P_\ell^x, \dots, P_{\ell_{max}-1}^x\} = \text{sa\_amg}(A_x, \mathbf{1}_{n_x})$  ▷ Build interpolation for velocity in  $x$ 
8:  $\{P_1^y, \dots, P_\ell^y, \dots, P_{\ell_{max}-1}^y\} = \text{sa\_amg}(A_y, \mathbf{1}_{n_y})$  ▷ Build interpolation for velocity in  $y$ 
9:  $\{P_1^p, \dots, P_\ell^p, \dots, P_{\ell_{max}-1}^p\} = \text{sa\_amg}(A_p, \mathbf{1}_{n_z})$  ▷ Build interpolation for pressure
10:  $\ell_{max} = \min(\ell_{max}^x, \ell_{max}^y, \ell_{max}^p)$  ▷ Calculate levels in grid hierarchy
11:
12: for  $\ell = 1, \dots, \ell_{max} - 1$  do
13:    $P_\ell = \begin{bmatrix} P_\ell^x & & \\ & P_\ell^y & \\ & & P_\ell^p \end{bmatrix}$  ▷ Assemble monolithic interpolation operator
14:    $K_{\ell+1} = P_\ell^T K_\ell P_\ell$  ▷ Compute coarse-grid operator
15: end for

```

Since we independently coarsen the velocity and pressure blocks, SA-AMG may determine different numbers of levels in each hierarchy. Thus, we form a block hierarchy using the fewest levels across these hierarchies. While this could lead to impractically large coarsest-grid systems if the coarsening of each field is dramatically different, we have yet to encounter any such problems. After the interpolation operators for each field are formed, they are combined as a block-diagonal matrix into a single monolithic interpolation operator P_ℓ and used in a monolithic calculation of the coarse-grid operator. We emphasize that we use SA completely independently on each field (lines 7, 8, and 9 in Algorithm 3.1), without any explicit coordination or coupling (in contrast to most traditional monolithic MG methods). As a result, the B block in K is projected to its coarse-grid analogue using both the velocity and pressure grid transfers. A natural concern is whether this independent coarsening might lead to somewhat strange stencils in B , where the coarse-grid discretization might not necessarily even be stable, even if the fine-level B is stable. While our approach does not guarantee stability, we have observed that we generally get stable coarse-grid operators as long as we maintain a similar ratio between the velocity and pressure DoFs throughout the hierarchy, which is crucial to the convergence of the overall AMG method. This can be achieved when we (approximately) match the coarsening rates for each field on each level. These coarsening rates are effectively determined by the strength-of-connection (SoC) measures used in aggregation, which identify the edges of the matrix graph that can be ignored during the coarsening process. Using traditional SoC measures, such as the symmetric SoC commonly used in SA [49], we found that achieving compatible coarsening rates was difficult, requiring significant tuning of drop-tolerance parameters for the velocity and pressure fields at each level of the hierarchy. Instead, we employ the so-called *evolution* SoC from [43], which provides more uniform coarsening by integrating local representations of algebraically smooth error into the SoC measures. We observe (experimentally) that this leads to consistent coarsening rates across the velocity and pressure fields, with no need for parameter tuning [51].

Remark 3.2 (applying Algorithm 3.1 directly to K_0). Algorithm 3.1 can be applied directly to the $\mathbb{P}_2/\mathbb{P}_1$ discretization, resulting in a preconditioner that we denote by HO-AMG. While the effectiveness of HO-AMG is expected to decline for discretizations with higher-order bases due to a mismatch in pressure and velocity field coarsening rates, we find that it offers stable performance for the $\mathbb{P}_2/\mathbb{P}_1$ discretization. However, direct application of Algorithm 3.1 to the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretization leads to poorly convergent or nonconvergent preconditioners, which we attribute to limitations in the algebraic coarsening of the discontinuous pressure field.

3.3. Relaxation. The final piece of the AMG method is the relaxation method with (weighted) error-propagation operator $I - \omega_\ell M_\ell^{-1} K_\ell$.

3.3.1. Additive Vanka. Vanka relaxation [48] is a standard relaxation scheme for saddle-point problems. It incorporates coupling between PDE components using local “patch” solves over small sets of DoFs (similar to small subdomains within an additive or multiplicative Schwarz method).

A Vanka patch is typically constructed to contain a small number of pressure DoFs (often just one) and all velocity DoFs that are adjacent (i.e., have nonzero connections) to this “seed” pressure DoF in the system matrix, K . Within geometric multigrid, such patches can be naturally constructed using topological arguments [20]; here, however, we construct the patches algebraically on each level of the hierarchy.

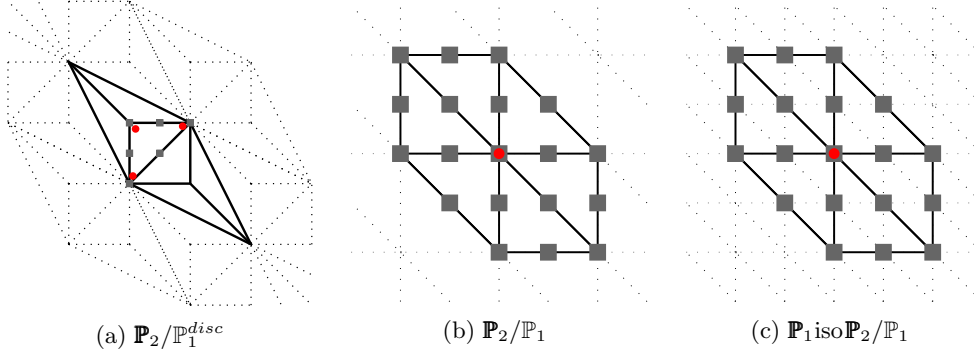


FIG. 2. (Left) one Vanka patch for the $\mathbf{P}_2/\mathbb{P}_1^{disc}$ discretization; (middle) one Vanka patch for the $\mathbf{P}_2/\mathbb{P}_1$ discretization; and (right) one Vanka patch for the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretization. The pressure DoF(s) (red circles, $\bullet$) and the velocity DoFs directly algebraically connected to that pressure DoF (gray squares, $\blacksquare$) are marked. Displayed patches are on 2D triangular meshes. (Color images are available online.)

Although the general mechanism for patch assembly is the same for all discretizations, we define each patch with a single seed pressure DoF for both the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ multigrid hierarchy and the fine-grid $\mathbf{P}_2/\mathbb{P}_1$ discretization. In these cases, each row of B corresponds to one patch, and patch i incorporates DoF i of the pressure and all velocity DoFs linked to nonzero entries in row i of B . For the $\mathbf{P}_2/\mathbb{P}_1^{disc}$ discretization, we employ three seed pressure DoFs for each patch, corresponding to the three pressure DoFs within each element. The three pressure DoFs on each element can be identified by three rows of B with identical sparsity patterns, making it possible to determine the seeds algebraically. However, since the information is readily available and requires minimal preprocessing, we opt to use a straightforward element-to-DoF map to identify the three pressure DoFs on a single element. Figure 2 depicts typical patches for the $\mathbf{P}_2/\mathbb{P}_1^{disc}$, $\mathbf{P}_2/\mathbb{P}_1$, and $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ discretizations of the fine-level operators. While these patches are directly linked to element/mesh entities on the finest mesh, a clear element/mesh interpretation is absent on algebraically coarsened levels.

On level ℓ , the i th patch matrix $K_\ell^{(i)}$ is then given by a square submatrix, which is composed of patch subblocks $B^{(i)}$ and $A^{(i)}$, written as

$$(3.6) \quad K_\ell^{(i)} = \begin{bmatrix} A^{(i)} & (B^{(i)})^T \\ B^{(i)} & 0 \end{bmatrix}.$$

Formally, $K_\ell^{(i)} = V_\ell^{(i)} K_\ell (V_\ell^{(i)})^T$, where $V_\ell^{(i)}$ is an injection operator defined for each patch, i , on level ℓ as an $m_\ell^{(i)} \times m_\ell$ binary matrix that extracts the subset of global DoFs on level ℓ coinciding with the i th patch, where m_ℓ is the total number of rows/columns of K_ℓ , while $m_\ell^{(i)}$ is the number of DoFs contained in $K_\ell^{(i)}$, which is one more than the number of velocity DoFs in the i th patch for the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ and $\mathbf{P}_2/\mathbb{P}_1$ discretizations, and three more for the $\mathbf{P}_2/\mathbb{P}_1^{disc}$ discretization.

In this work, we use the additive form of Vanka relaxation [46, 19]. A single Vanka sweep for $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbb{P}_1$ and $\mathbf{P}_2/\mathbb{P}_1$ discretizations is given by

$$M_\ell^{-1} = \sum_{i=1}^{n_\ell^{(p)}} (V_\ell^{(i)})^T W_\ell^{(i)} \left(V_\ell^{(i)} K_\ell (V_\ell^{(i)})^T \right)^{-1} V_\ell^{(i)},$$

where the diagonal weighting matrix, $W_\ell^{(i)}$, is a partition of unity, with each diagonal entry equal to the reciprocal of the number of patches that contain the associated DoF. For the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretization, the sum extends over fewer patches, for $1 \leq i \leq n_\ell^{(p)}/3$.

Applying inverses of patch matrices often constitutes the most computationally demanding part of the multigrid method. The direct inverse of the patch can be applied using highly optimized dense-linear algebra kernels. However, this requires significant storage, as it results in a fully dense inverse matrix over each patch. A sparse LU factorization reduces the memory footprint but results in slower solution times due to sparse forward and backward substitutions. Our performance studies, not presented here, show that sparse LU does not outperform the direct-inverse method in efficiency. However, the dense inverses' high storage may be limiting.

Therefore, we introduce a novel approach that balances the effectiveness of dense-linear algebra with reduced storage for sparse LU factorizations. Instead of computing the inverse of each $K_\ell^{(i)}$, we leverage a block LU factorization of each Vanka patch,

$$(3.7) \quad K_\ell^{(i)} = \begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix} = \begin{bmatrix} A & \\ B & S \end{bmatrix} \begin{bmatrix} I & U \\ & I \end{bmatrix},$$

with sub- and superscripts on the subblocks omitted for clarity. Here, $U = A^{-1}B$, and $S = -BA^{-1}B^T$ denotes the Schur complement. We emphasize that B^T and U have small column dimension (one in the typical case, but three for the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretization) and that S is a small matrix. The action of the inverse for $K_\ell^{(i)}$ is

$$(3.8) \quad K_\ell^{(i)} x_\ell^{(i)} = b_\ell^{(i)} \Rightarrow x_\ell^{(i)} = \begin{bmatrix} I & \hat{U} \\ & I \end{bmatrix} \begin{bmatrix} A^{-1} & \\ \hat{B} & S^{-1} \end{bmatrix} b_\ell^{(i)},$$

where $\hat{B} = -S^{-1}BA^{-1}$, $\hat{U} = -U$, and $\hat{B}$, $\hat{U}$, and S^{-1} are precomputed in the setup phase. We further leverage the block-diagonal nature of the vector Laplacian operator to minimize the storage associated with the inverse of subblock A up to a factor of 2 and 3 in two dimensions and three dimensions, respectively. Furthermore, this method reduces the number of floating-point operations necessary for calculating the action of the patch's inverse, thereby enhancing the computational efficiency of the setup and solution phases.

Vanka relaxation exhibits sensitivity to its damping parameter, ω_ℓ . Fourier analysis is often used to determine ω_ℓ when Vanka is employed within a geometric multigrid scheme [26, 19, 36, 11]; however, this method is not feasible for the irregular coarse meshes generated in an AMG algorithm. Several recent papers have used alternative methods to tune relaxation parameters. In [1], for example, instead of a stationary weighted relaxation scheme, a Chebyshev iteration is used, with hand-tuned Chebyshev intervals chosen to optimize the resulting solver performance. While this approach is, in principle, also applicable here, it could require significant tuning for the various levels of the multigrid hierarchy. Instead, we opt for a more practical approach where relaxation on each level is accelerated using FGMRES.¹ FGMRES does not necessarily provide optimal relaxation weights as it constructs a residual-minimizing polynomial, which is not the same as optimally damping high-frequency error. While it does not provide general convergence bounds as can sometimes be achieved using Chebyshev iterations, it does eliminate the need for additional tuning

¹We use FGMRES instead of classical GMRES to avoid the extra application of a (possibly expensive) preconditioner needed by classical GMRES to recover the approximate solution.

parameters and generally provides satisfactory damping weights. One minor drawback is that FGMRES requires additional inner products, though this is a minimal computational cost over the Vanka relaxation patch solves.

In the remainder of the paper, we fix the relaxation to include two inner FGMRES iterations with one Vanka relaxation sweep per iteration, akin to setting $\nu_1 = \nu_2 = 2$ in (3.1) or (3.4). Results below show that using this Krylov-wrapping approach on all levels in the hierarchy provides stable and robust convergence for systems discretized with $\mathbb{P}_2/\mathbb{P}_1$ elements. However, we observed stagnant convergence when applied to systems discretized with $\mathbb{P}_2/\mathbb{P}_1^{disc}$ elements. To address this issue, we instead use a stationary Vanka iteration on the finest grid with the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ elements in (3.1) and Krylov acceleration on all other levels in (3.4). We perform a parameter search to identify the optimal ω_0 value, denoted as ω_0^{opt} , for the K_0 systems discretized with the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ element pair, discussed below.

3.3.2. Distributive least-squares commutator. We consider a distributive least-squares commutator (DLSC) method that builds on the LSC-DGS approach outlined in [54]. LSC-DGS has shown to be effective for various Stokes problems, including its adaptation to linearized Navier–Stokes equations [15] and as a DC preconditioner in low-order Stokes problems [54]. The LSC-DGS method is derived from Schur complement approximations using a least-squares-based approximate commutator. The notable advantage is the elimination of user-defined parameters, unlike in the classical DGS method. For an in-depth study of the LSC-DGS method in the context of the Marker and Cell discretization, readers are referred to [54].

DLSC methods rely on the commutative assumption that, under certain conditions, $AM_u^{-1}B^T \approx B^T M_p^{-1}A_p$, where A_p represents a discrete Laplacian on the pressure, and M_u and M_p are velocity and pressure mass matrices, respectively. This assumption leads to an approximate factorization, which can be expressed as

$$\begin{aligned} \begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} M_u^{-1} & M_u^{-1}B^T \\ 0 & -M_p^{-1}A_p \end{bmatrix} &\approx \begin{bmatrix} AM_u^{-1} & 0 \\ BM_u^{-1} & BM_u^{-1}B^T \end{bmatrix}, \\ \text{or } \begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix}^{-1} &\approx \begin{bmatrix} M_u^{-1} & M_u^{-1}B^T \\ 0 & -M_p^{-1}A_p \end{bmatrix} \begin{bmatrix} AM_u^{-1} & 0 \\ BM_u^{-1} & BM_u^{-1}B^T \end{bmatrix}^{-1}. \end{aligned}$$

Further simplifications are achievable by setting

$$A_p = M_p \hat{A}_p = M_p (BM_u^{-1}B^T)^{-1} (BM_u^{-1}AM_u^{-1}B^T)$$

and factorizing the lower-triangular matrix:

$$\begin{aligned} \begin{bmatrix} M_u^{-1} & M_u^{-1}B^T \\ 0 & -M_p^{-1}(M_p \hat{A}_p) \end{bmatrix} \begin{bmatrix} M_u & \\ & I \end{bmatrix} \begin{bmatrix} A & 0 \\ B & BM_u^{-1}B^T \end{bmatrix}^{-1} \\ \begin{bmatrix} I & M_u^{-1}B^T \\ 0 & -\hat{A}_p \end{bmatrix} \begin{bmatrix} A & 0 \\ B & BM_u^{-1}B^T \end{bmatrix}^{-1}. \end{aligned}$$

The DLSC iteration for the linear system

$$\begin{bmatrix} A & B^T \\ B & 0 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} = \begin{bmatrix} \mathbf{f} \\ g \end{bmatrix}$$

is then given by

$$(3.11) \quad \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} \leftarrow \begin{bmatrix} \mathbf{u} \\ p \end{bmatrix} + \begin{bmatrix} I & M_u^{-1}B^T \\ 0 & \hat{A}_p \end{bmatrix} \begin{bmatrix} A & 0 \\ B & BM_u^{-1}B^T \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{f} - A\mathbf{u} - B^T p \\ g - B\mathbf{u} \end{bmatrix}.$$

Algorithm 3.2 DLSC.

```

1: Input:  $\mathbf{u}$ ,  $p$ , initial guess
2:        $\mathbf{f}$ ,  $g$ , components of the right-hand side
3: Output:  $\mathbf{u}^{\text{new}}$ ,  $p^{\text{new}}$ , updated variables
4:
5:  $\mathbf{v}_* \leftarrow$  Approx. solve  $A\mathbf{v}_* = \mathbf{f} - A\mathbf{u} - B^T p$   $\triangleright$  solve the momentum equation
6:  $q \leftarrow$  Approx. solve  $BM_u^{-1} B^T q = g - B(\mathbf{u} + \mathbf{v}_*)$   $\triangleright$  solve the continuity equation
7:
8:  $\delta p \leftarrow$  Approx. solve  $BM_u^{-1} B^T \delta p = -(BM_u^{-1} A M_u^{-1} B^T) q$   $\triangleright$  pressure correction
9:  $\delta \mathbf{u} = \mathbf{v}_* + M_u^{-1} B^T q$   $\triangleright$  velocity correction
10:
11:  $\mathbf{u}^{\text{new}} = \mathbf{u} + \delta \mathbf{u}$   $\triangleright$  velocity update
12:  $p^{\text{new}} = p + \delta p$   $\triangleright$  pressure update

```

Notably, operator M_p does not appear in the final form of the algorithm as seen in (3.11) and Algorithm 3.2. In the context of this work, unless stated otherwise, M_u is replaced by the diagonal of the mass matrix. This mass matrix scaling is consistently applied across all $\mathbb{P}_2/\mathbb{P}_1$ grids. For the coarse-level velocity mass matrix, we utilize the Galerkin product, constructed using the grid transfer operators from Algorithm 3.1. On $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ grids, the velocity mass matrix is approximated by an identity operator, as using a diagonal mass matrix approximation does not meaningfully impact the convergence.

Algorithm 3.2 describes a single sweep of this relaxation, where the first two solve steps (lines 5 and 6) result from the block lower-triangular forward solve and where the last solve and correction (lines 8 and 9) results from the block upper triangular matrix multiplication in (3.11). A central aspect in the LSC-DGS algorithm of [54] is the use of Gauss–Seidel to approximate the solutions to the A and $BM_u^{-1} B^T$ matrix systems. In contrast, we make use of Chebyshev polynomial relaxation (and, later, a standalone AMG V-cycle), which leads to improved parallel efficiency over Gauss–Seidel. Our empirical observations for DLSC, which are largely in agreement with the findings in [54], reveal that the pressure-correction step (line 8) typically requires more iterations compared to the momentum relaxation step (line 5) and the continuity relaxation step (line 6). This requirement becomes increasingly pronounced in the context of anisotropic unstructured meshes and 3D problems. This loosely translates to the fine-grid velocity being closer to being divergence-free when the pressure-correction step is solved more accurately.

Numerical results in section 4 show that this is an effective relaxation scheme for the $\mathbb{P}_2/\mathbb{P}_1$ and $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ discretizations, but we did not find it to be effective for the $\mathbb{P}_2/\mathbb{P}_1^{\text{disc}}$ discretization. We note that [54] proposes the use of elementwise block Gauss–Seidel relaxation for $BM_u^{-1} B^T$ in a similar setting; block variants such as block-Jacobi–Chebyshev, either standalone or within an AMG V-cycle, may also be effective, but we leave this open question to future work. In all SV results in section 4, we only use cellwise Vanka relaxation (and not DLSC) for the $\mathbb{P}_2/\mathbb{P}_1^{\text{disc}}$ discretization.

4. Numerical results. We compare the HO-AMG preconditioner with the proposed DC AMG preconditioner using two different relaxation types: additive Vanka and DLSC. The HO-AMG preconditioner is constructed by applying Algorithm 3.1 directly to the high-order discretization (K_0), while DC applies Algorithm 3.1 to an auxiliary operator, K_1 . For the DC preconditioners, we consider three different

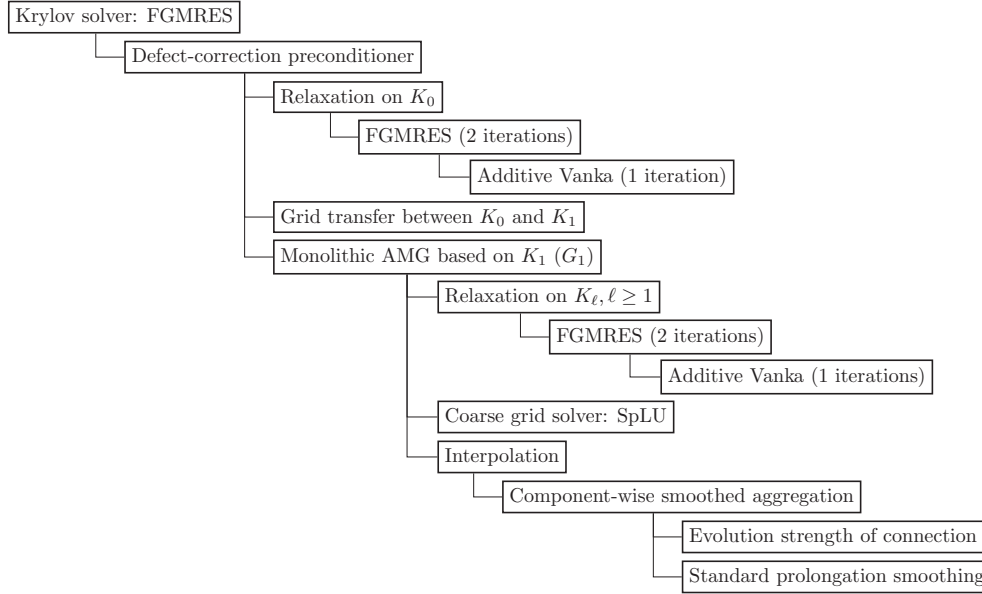


FIG. 3. Solver diagram for DC preconditioners. If relaxation is performed on all levels, the diagram corresponds to the DC-all solver.

relaxation schedules: DC-all, DC-skip1, and DC-skip0. The DC-all preconditioner performs pre- and postrelaxation on all levels of the hierarchy, except for the coarsest level (n_{lvs}), where an exact solution is used. DC-skip1 skips relaxation on the K_1 system but performs relaxation on the K_0 system and remaining levels of the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbf{P}_1$ hierarchy, except for a direct solve on the coarsest level. DC-skip0 skips relaxation on the K_0 system and performs pre- and postrelaxation on remaining levels, except for a direct solve on the coarsest level. A solver diagram for the DC approach using Vanka relaxation is given in Figure 3. HO-AMG is similar (incorporating our AMG enhancements) except that the Krylov solver is directly applied to monolithic AMG based on K_0 . We will also explore additional preconditioning variants (described later) based on the DC operator. The notation for all relevant parameters and solvers is summarized in Table 1.

In the DC preconditioners using DLSC, we perform two iterations of DLSC without Krylov acceleration during relaxation. The optimal degree of the Chebyshev polynomial depends on the stage of Algorithm 3.2 and the mesh type, as outlined in Table 2. For 3D meshes, higher Chebyshev polynomial degrees are necessary to achieve convergence rates similar to Vanka relaxation. We note that the pressure-correction step is often the limiting factor for convergence, particularly for 3D problems, where higher polynomial degrees are needed to achieve convergence rates comparable to 2D cases.

As a point of comparison for the monolithic AMG preconditioners, we consider a block-triangular (or inexact Uzawa) preconditioner, denoted by BTP. This preconditioner approximates the lower-triangular factor in the block factorization of the Stokes operator [16, 41]. It computes the velocity and pressure updates, $[\delta \mathbf{u}, \delta p]$, as follows:

$$\begin{aligned}\delta \mathbf{u} &= Q_A^{-1} \mathbf{r}_u \\ \delta p &= Q_B^{-1} (B \delta \mathbf{u} - r_p),\end{aligned}$$

TABLE 1
Notation reference table.

Symbol	Description
K_0	saddle-point system for the higher-order discretizations ($\mathbb{P}_2/\mathbb{P}_1$, $\mathbb{P}_2/\mathbb{P}_1^{disc}$)
K_1	saddle-point system for the lower-order discretization ($\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$)
G_1	monolithic AMG Solver based on K_1
ℓ	generic level in the monolithic multigrid hierarchy
n_{lvl}	total number of levels/grids in the monolithic multigrid hierarchy
η_u	momentum equation residual scaling parameter on level 0
η_p	continuity equation residual scaling parameter on level 0
η	K_0 residual scaling parameters when $\eta_u = \eta_p$
ω_ℓ	Vanka relaxation damping parameter
Superscript <i>opt</i>	denotes that the optimal parameter was computed via brute-force search
BTP	inexact block triangular preconditioner based on K_0
HO-AMG	monolithic AMG algorithm applied directly on K_0
DC-all	DC with relaxation on all levels, $\omega_\ell = 1$ for $\ell \in [0, n_{lvl})$
DC-skip0	DC with relaxation on all but the K_0 level, $\omega_\ell = 1$ for $\ell \in [1, n_{lvl})$
DC-skip1	DC with relaxation on all but the K_1 level, $\omega_\ell = 1$ for $\ell \in [0] \cup [2, n_{lvl})$
——— Line	DC preconditioner with Vanka relaxation on $\ell \in [0, n_{lvl})$
..... Line	DC preconditioner with DLSC relaxation on $\ell \in [0, n_{lvl})$
- - - - - Line	DC preconditioner with Vanka on $\ell = 0$ and DLSC on $\ell \in [1, n_{lvl})$

TABLE 2

Degrees of Chebyshev polynomials used for polynomial relaxation at different stages of the DLSC algorithm (Algorithm 3.2). These degrees vary depending on the dimension (2D or 3D) of the problem. Chebyshev polynomials on the interval $[0.5^d\lambda, 1.1\lambda]$ are used, where d represents the problem's dimension and λ is an estimate of the largest eigenvalue of A or $BM_u^{-1}B^T$ generated by an Arnoldi iteration.

Update stage	2D	3D
Momentum equation, Algorithm 3.2 line 5	3	4
Continuity equation, Algorithm 3.2 line 6	3	4
Pressure correction, Algorithm 3.2 line 8	6	16

where $[\mathbf{r}_u, \mathbf{r}_p] = \mathbf{b} - \mathbf{K}\mathbf{x}$, Q_A^{-1} represents an approximation of the vector Laplacian inverse, and Q_B^{-1} denotes an exact or approximate inverse of the pressure-mass matrix, depending on the context. To approximate Q_A^{-1} , we employ a single V-cycle of SA-AMG. For Q_B^{-1} , we use either an exact or approximate mass matrix inverse, as outlined in [16]. The approach to Q_B^{-1} is influenced by the discretization used. In the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ case, Q_B is block-diagonal, enabling us to compute the exact inverse at low cost. However, when using the $\mathbb{P}_2/\mathbb{P}_1$ discretization, we approximate the action of Q_B^{-1} using a single V-cycle of SA-AMG. The SA-AMG solvers are assembled using evolution SoC, standard aggregation, and standard prolongator smoothing for interpolation. For multigrid relaxation, we employ two steps of Krylov-wrapped Jacobi relaxation.

The $\mathbb{P}_2/\mathbb{P}_1$, $\mathbb{P}_2/\mathbb{P}_1^{disc}$, and $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ systems are assembled in Firedrake [25, 32]. Although Firedrake currently does not provide the capability to construct $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ matrices directly, we form them by assembling the $\mathbb{P}_1/\mathbb{P}_1$ discretization on a refined mesh followed by geometric restriction to a coarsened pressure field. This indirect approach to forming the $\mathbb{P}_1\text{iso}\mathbb{P}_2/\mathbb{P}_1$ discretization can be avoided in a finite-element code base that supports macroelements. In our open-source implementation (<https://github.com/Alexey-Voronin/Monolithic-AMG-For-Stokes>), we use the PyAMG library [5] to implement AMG preconditioners that combine Algorithm 3.1 with Vanka relaxation methods.

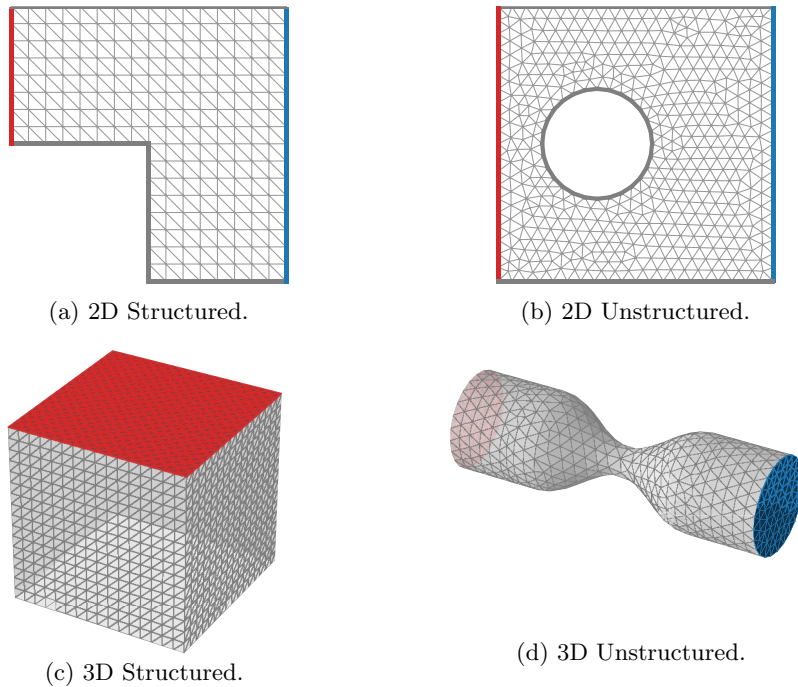


FIG. 4. Meshes used to construct the Stokes problems. The boundaries are marked with three colors (red, blue, and gray), corresponding to three types of velocity field BCs. Red corresponds to nonzero Dirichlet boundaries. Blue corresponds to Neumann BCs. Gray indicates zero Dirichlet BCs.

We consider both structured and unstructured meshes in two dimensions and three dimensions for our numerical solvers study. The structured meshes are generated using the Firedrake software, while the unstructured meshes are generated using Gmsh [23]. Barycentric refinement of meshes uses the code associated with [21]. As test problems, we consider the following, with domains pictured in Figure 4.

Structured 2D: Backward-Facing Step. A standard 2D backward-facing step domain, discretized with structured triangular elements [16]. The left domain surface is marked in red in Figure 4, where a parabolic inflow BC is imposed. (Color figures are available online.) The right domain surface, marked in blue, has a natural (Neumann) BC. All other domain surfaces (marked in grey) have zero Dirichlet BCs on the velocity field.

Structured 3D: Lid-Driven Cavity. A standard lid-driven cavity problem on a 3D domain, discretized with structured tetrahedral elements [16]. The top face of the domain, marked in red, has a Dirichlet BC imposed with constant tangential velocity, while all other faces have zero Dirichlet BCs on the velocity field.

Unstructured 2D: Flow Past a Cylinder. This test problem simulates flow past a cylinder on a 2D domain, discretized with unstructured triangular elements. The left domain surface, marked in red, has a parabolic inflow BC imposed, while the right surface, marked in blue, has a natural outflow BC. All other boundaries including those on the interior boundary of the cylinder, have zero Dirichlet BCs on the velocity field.

Unstructured 3D: Pinched Channel This test problem simulates flow through a pinched tube on a 3D domain, discretized with unstructured tetrahedral elements. The left face of the domain, marked in red, has a constant velocity inflow BC, while the right face, marked in blue, has a natural outflow BC. All other faces of the domain have zero Dirichlet BCs on the velocity field.

For each problem, we present iteration counts, relative solution times, and relative per iteration times. Additionally, we include the total relative times, including the setup costs, for the two discretization types (see Figures 8 and 10). The iteration counts are the total number of preconditioned FGMRES iterations required to reduce the ℓ_2 norm of the residual by a relative factor of 10^{10} . For all tests, we use restarted FGMRES with a maximum Krylov subspace size of 20. The timings are measured relative to the BTP preconditioner. We report the shortest of five independent time-to-solution measurements to account for time measurement variability.

To find the best damping parameters, η and ω_0 , we use a brute-force search for each problem type and multigrid variant. For the $\mathbf{P}_2/\mathbb{P}_1$ discretization, we only find η , because Krylov-wrapping avoids the use of ω_ℓ . However, for the $\mathbf{P}_2/\mathbb{P}_1^{disc}$ discretization, Krylov-wrapping on $\ell = 0$ is ineffective. Therefore, for the $\mathbf{P}_2/\mathbb{P}_1^{disc}$ DC-all preconditioner, we calculate both η and ω_0 . We define the optimal parameter set as the value(s) that result in the fewest number of preconditioned FGMRES iterations required to reach a relative reduction in the ℓ^2 -norm of the residual by a factor of 10^8 . All parameter searches were performed on problems with at least one million DoFs and four (three) levels in the G_1 multigrid hierarchy in two dimensions (three dimensions). Our results reveal that even a limited search yields near-optimal parameters, indicating low sensitivity to small parameter variations. In cases where multiple sets of parameters result in the same number of iterations, we select the optimal set of parameters to be the one that has the smallest measured convergence factor, determined by finding the geometric mean of the residual reduction factors during the last 10 FGMRES iterations. In the cases where the optimal parameter range includes value 1.0, we select it as the optimal parameter value for simplicity.

4.1. Preconditioning $\mathbf{P}_2/\mathbb{P}_1$ systems. The optimal damping parameter values, η^{opt} , used within this section are shown in Table 3. To understand this parameter choice, we examine the effect of η on the convergence rate of the FGMRES solver when using the four (three) level AMG hierarchy preconditioner for 2D (3D) problems with at least one million DoFs on the $\mathbf{P}_1$ iso $\mathbf{P}_2/\mathbb{P}_1$ grid. The DC-skip preconditioner does not require the selection of η , as the FGMRES iterations are invariant to changes in scaling and, in this case, η simply scales the entire preconditioner. Our results,

TABLE 3
Optimal damping parameters $\eta^{opt} = \eta_u^{opt} = \eta_p^{opt}$ for Vanka and DLSC relaxation in TH, $\mathbf{P}_2/\mathbb{P}_1$ preconditioners. DC-skip0 results for DLSC are excluded due to poor performance across all problems.

Relaxation	Preconditioner	Mesh type			
		Structured		Unstructured	
		2D	3D	2D	3D
Vanka	DC-skip1	0.86	1.00	0.75	1.00
	DC-skip0	1.00	1.00	1.00	1.00
	DC-all	0.75	1.00	0.80	1.00
DLSC	DC-skip1	1.00	0.60	1.00	1.00
	DC-all	1.00	1.10	1.00	1.06

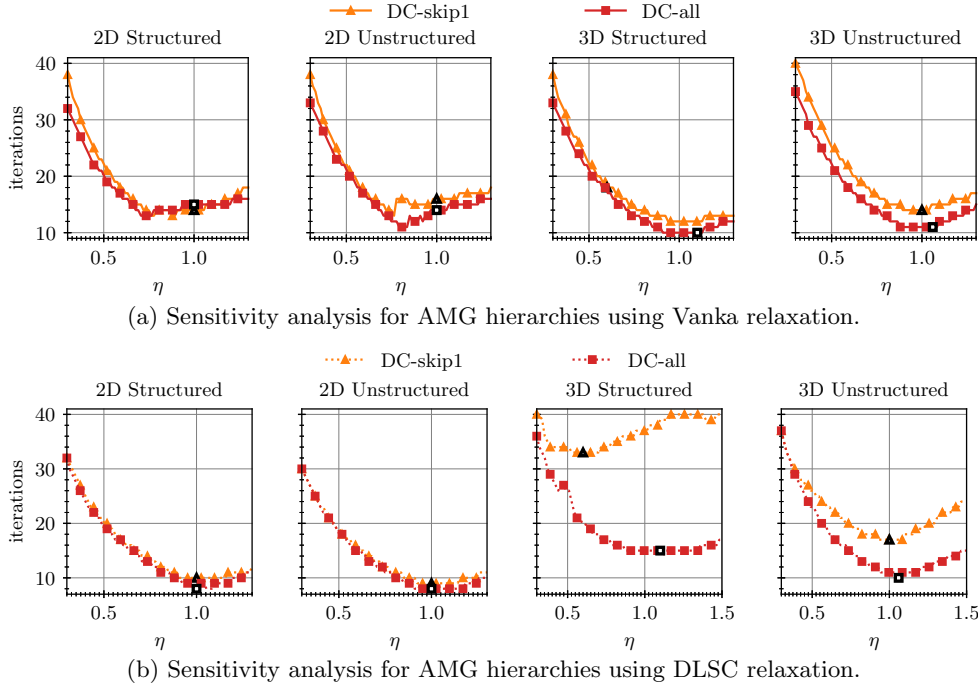


FIG. 5. Sensitivity analysis of the measured iterations to convergence for the AMG preconditioned FGMRES solvers with Vanka and DLSC relaxation. Optimal parameter values, η^{opt} , are outlined in black (every fifth sample is marked).

as shown in Figure 5, indicate that the optimal damping parameters for structured and unstructured 2D problems using Vanka fall within the range of 0.75–0.86. The results for 3D problems using Vanka show a steady plateau where the minimum iteration count occurs, with the optimal η parameter for all 3D problems selected as 1.0, which falls within this interval. In contrast, for DLSC, we find $\eta = 1.0$ is optimal for 2D problems, but variations are seen for 3D problems. Importantly, across all cases, underestimating this optimal parameter results in quicker performance degradation than overestimating it.

Using the above-described parameter choices, we examine the convergence of FGMRES preconditioned with the monolithic AMG-based preconditioners based on Vanka relaxation and compare them against the BTP preconditioner for a range of problems in Figure 6. For most problems, the HO-AMG preconditioner (based on the $\mathbb{P}_2/\mathbb{P}_1$ AMG hierarchy) exhibits a slight growth in the iteration count as the problems are refined. Similar slight growth is seen for the DC approach based on the $\mathbb{P}_{1iso}\mathbb{P}_2/\mathbb{P}_1$ discretization; however, DC-skip1, which effectively replaces the coarse grids of the HO-AMG hierarchy with an AMG hierarchy based on K_1 , consistently converges in 1–3 fewer iterations than HO-AMG. Since the DC-skip1 coarse grids are sparser than those of HO-AMG, the relative cost of the solution is slightly cheaper, as can be seen in the timing rows of Figure 6. However, the solution times are similar for HO-AMG and DC-skip1, because the relaxation cost at the finest level dominates the time to solution. For DC-skip0, relaxation on the K_0 system is skipped and replaced by relaxation on the K_1 system. The convergence of DC-skip0 suffers for all problem types, as seen in the top row of Figure 6. Despite having a lower cost per iteration than HO-AMG, the increase in the iteration count makes DC-skip0 less

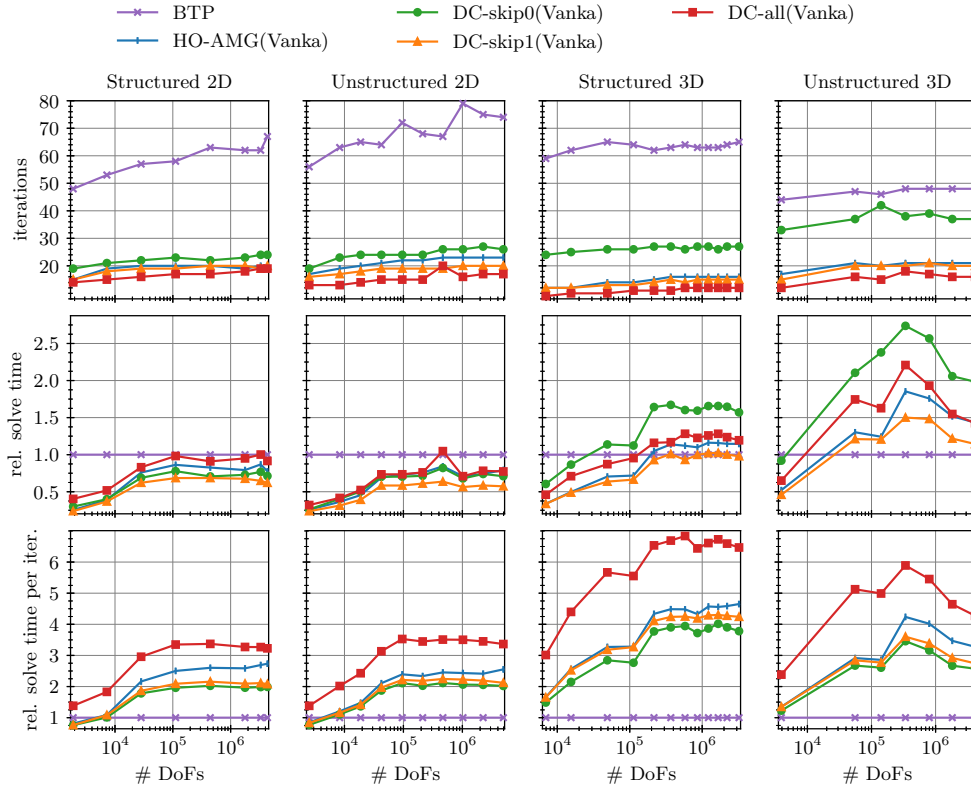


FIG. 6. $\mathbb{P}_2/\mathbb{P}_1$ Vanka relaxation-based monolithic AMG solvers. Iterations to convergence (top), relative time to solution (middle), and relative time per iteration (bottom). The timings are relative to the BTP solver.

computationally effective than DC-skip1 and HO-AMG. This effect is particularly drastic for 3D problems, which can be seen in the last two columns of Figure 6. Although the solver based on DC-all converges in the fewest number of iterations for most problems, doubling the number of relaxation sweeps on fine grids (relaxing on both K_0 and K_1) results in significantly longer times to solution than for the HO-AMG approach. However, the relative difference in timing between DC-all and HO-AMG appears to diminish with larger problem sizes.

The convergence and timing results in Figure 6, especially for DC-skip0 and DC-skip1, show conclusively that relaxation on the original problem (K_0) should not be skipped when designing a DC type AMG preconditioner. Additional relaxation on the K_1 system may improve convergence further but at a higher computational cost. This conclusion is supported by the results from a geometric multigrid-based DC approach published in [50].

In contrast, the BTP preconditioners exhibit slower convergence, requiring approximately 3–5 times more iterations to reach the convergence tolerance. This effect is particularly noticeable for 2D problems, where the iteration counts show significant growth. Comparing the time to convergence, we find that the monolithic AMG solvers outperform the BTP-based solvers by 20–50% for 2D problems. For 3D problems, however, BTP is generally the fastest to converge, although DC-skip1 offers similar performance on structured grids. This difference in performance is partly due to the reduced number of iterations required for convergence of BTP in three

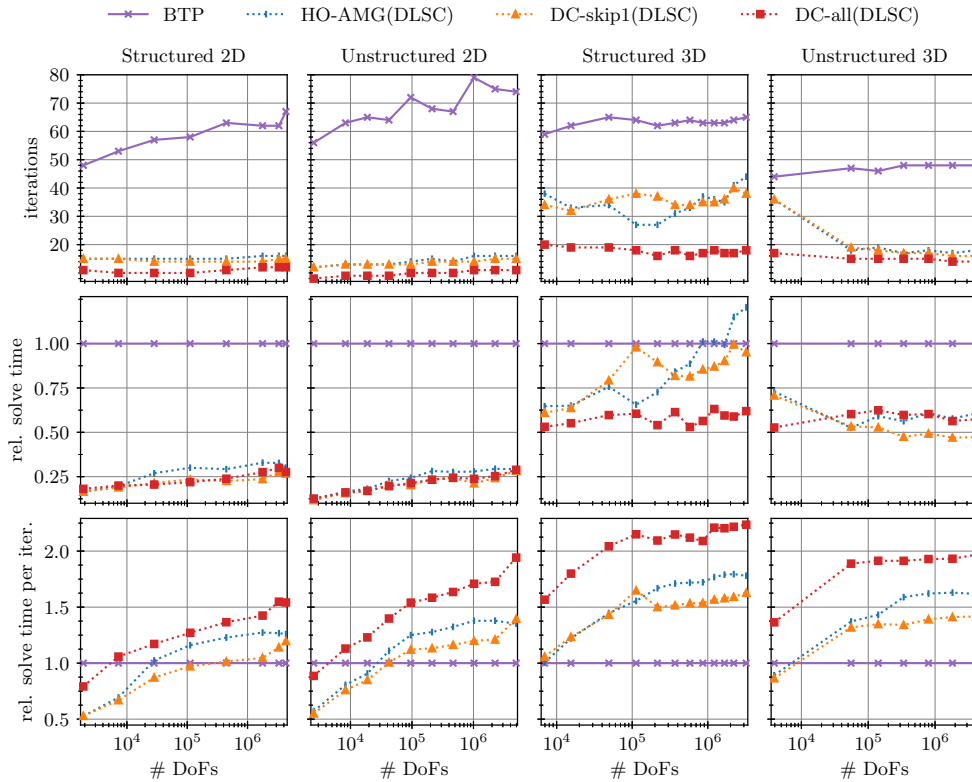


FIG. 7. $\mathbf{P}_2/\mathbf{P}_1$ DLSC relaxation-based monolithic AMG solvers. Iterations to convergence (top), relative time to solution (middle), and relative time per iteration (bottom). The timings are relative to the BTP solver.

dimensions, coupled with the larger Vanka patch sizes at each hierarchy level, underscoring the need for further research on implementation efficiencies.

We next consider the performance of FGMRES preconditioned with DLSC-based monolithic AMG preconditioners, continuing to compare with BTP. The timing results shown in Figure 7 include results for BTP identical to those in Figure 6, which helps evaluate the effectiveness of Vanka versus DLSC-based AMG solvers. As seen in the top row of Figure 7, the number of iterations required for the DLSC-based preconditioners follows similar trends to those observed with Vanka-based AMG solvers. Specifically, the DC-all preconditioners achieve convergence in fewer iterations than both DC-skip1 and HO-AMG. The performance of DC-skip1 and HO-AMG in terms of iterations is similar across various problem types. A notable difference between Vanka and DLSC-based AMG preconditioners is the larger increase in iteration count for 3D structured problems in the latter, even after applying additional Chebyshev sweeps as detailed in Table 2.

The middle section of Figure 7 presents an intriguing finding regarding time-to-solution. For 2D problems, DLSC-based AMG solvers are 75–80% faster than BTP, converging much more quickly. In 3D scenarios, this lead diminishes to about 25–50% for DC-skip1, suggesting an average solution speed about 1.3–2 times faster than Vanka-based AMG solvers. Despite similar iteration counts, the DC solver DC-skip1 is found to be consistently more efficient than HO-AMG, thanks to its sparser coarser

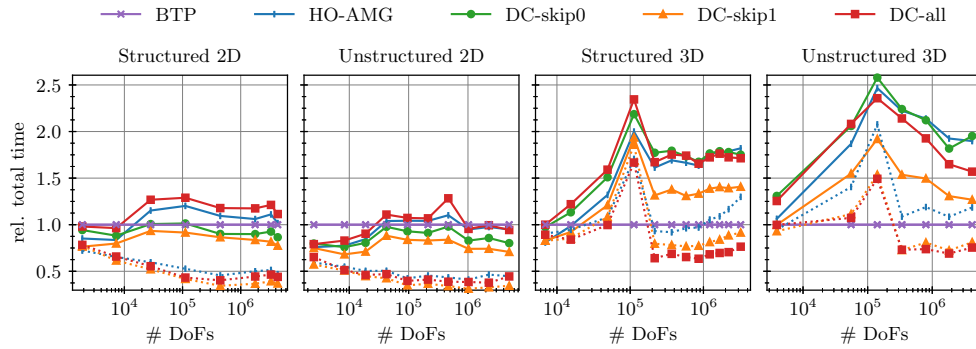


FIG. 8. Relative total time to solution, including multigrid setup and solve phases, for $\mathbf{P}_2/\mathbf{P}_1$ problems. The solid HO and DC lines correspond to solvers utilizing Vanka relaxation, and dashed lines correspond to LSC-DGS relaxation. The timings are relative to the BTP solver.

operators. The final rows of Figures 6 and 7 indicate that, per iteration, DLSC-based AMG preconditioners are more cost-effective than their Vanka-based counterparts.

The findings from Figures 6 and 7 underscore the importance of including relaxation on the original problem (K_0) in DC type AMG preconditioners. Adding relaxation for K_1 improves convergence but increases computational costs.

Turning to the total time to solution, which includes both multigrid setup and solve phases, Figure 8 summarizes the performance of all solver types described thus far. The plots clearly show that the LSC-DGS-based multigrid methods (dashed lines) are consistently faster than Vanka-based solvers (solid lines), a difference attributed to their faster setup and solve phases. For 2D problems, most Vanka-based solvers and all LSC-DGS-based solvers outperform BTP. However, in the 3D case where the cost of Vanka assembly and application rises drastically, none of the Vanka-based solvers can outperform BTP for large problem sizes. In contrast, the LSC-DGS-based solvers outperform BTP by 10–30% for most large 3D problem sizes. Overall, for all problems considered, DLSC-based AMG preconditioners offer significant advantages in speed and cost-effectiveness over Vanka-based methods.

4.2. Preconditioning $\mathbf{P}_2/\mathbb{P}_1^{disc}$ Systems. In this section, we evaluate the effectiveness of the DC preconditioner for solving systems that have been discretized using $\mathbf{P}_2/\mathbb{P}_1^{disc}$ elements on 2D structured and unstructured meshes. Preliminary numerical experiments (not reported here) showed that relaxation on all levels of the AMG hierarchy is needed to generate an efficient solver. The convergence data in Figure 10 uses the optimal damping parameter values tabulated in Table 4.

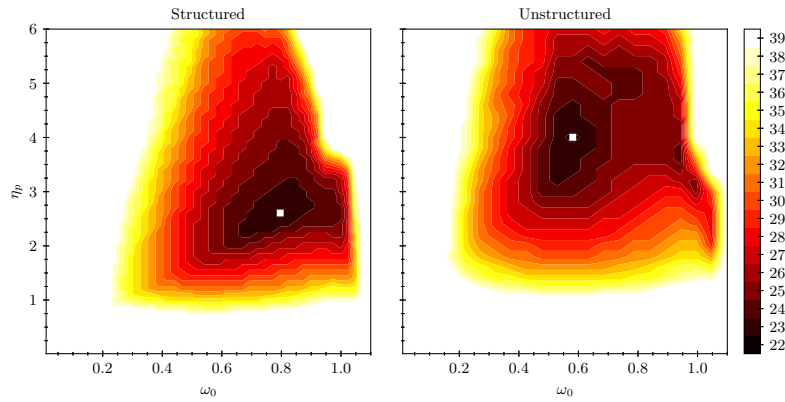
We study the sensitivity of the FGMRES convergence rate to the parameter choices in DC-all by performing a parameter search on 3-level AMG hierarchies with at least a quarter million DoFs on the finest $\mathbf{P}_1$ iso $\mathbf{P}_2/\mathbf{P}_1$ grid for both structured and unstructured problems. We explore two DC-all DC strategies: the first applying Vanka relaxation across all levels except the coarsest, while the second, termed “Vanka-DLSC,” utilizes Vanka relaxation at the finest level ($\ell = 0$) and DLSC on coarser levels ($\ell > 0$). Both strategies adhere to the relaxation sweep schedules previously discussed.

We found that for both relaxation types $\eta_u \approx 1$ produced the best convergence results, regardless of the values of ω_0 and η_p , and so we fix $\eta_u^{opt} = 1$. To determine the other parameters, we perform a 2D parameter scan. The 2D heat maps in Figure 9 show the iteration counts for different choices of ω_0 and η_p . The results indicate that

TABLE 4

Optimal parameters for $SV, \mathbf{P}_2/\mathbb{P}_1^{disc}$, DC-all preconditioners: overweighting η_p^{opt} , and damping ω_0^{opt} .

Relaxation	Mesh Type	Parameters	
		ω_0^{opt}	η_p^{opt}
Vanka($\ell \geq 0$)	Structured	0.78	2.68
	Unstructured	0.58	4.00
Vanka($\ell = 0$), DLSC($\ell > 0$)	Structured	0.87	2.96
	Unstructured	0.58	3.80



(a) AMG preconditioners using Vanka relaxation.

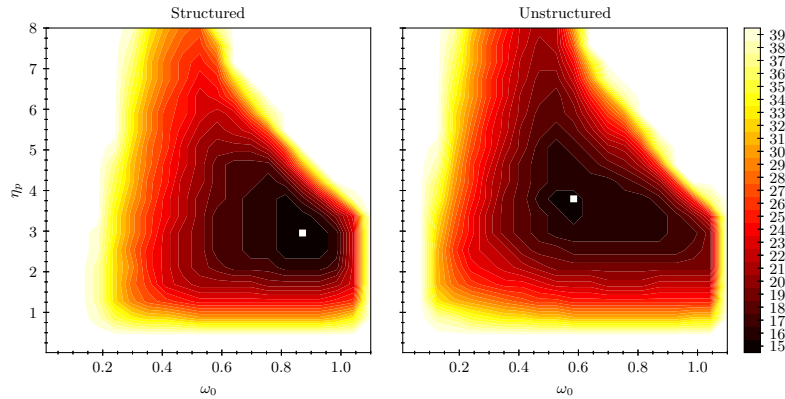
(b) AMG preconditioners using Vanka on $\ell = 0$ and DLSC on $\ell > 0$ multigrid levels.

FIG. 9. Sensitivity analysis of iterations to convergence for DC-all preconditioned FGMRES solvers. Optimal parameters are highlighted with white squares.

the iteration counts are relatively insensitive to the choice of ω_0 as long as they are between 0.5 and 0.9, and a similar trend is observed for the overweighting parameter, η_p , where values between 1.8 and 5 result in the best convergence. The optimal convergence regions for structured and unstructured problems are relatively close, but unstructured problems tend to perform best with smaller ω_0 and larger η_p values than structured problems. The optimal parameter choices are summarized in Table 4.

Using the parameter choices described above, we now examine the convergence of these solvers across a range of problems and problem sizes. The iteration counts in

Figure 10 show that both DC-all (Vanka) and DC-all (Vanka-DLSC) preconditioners converge, on average, in 27 iterations for structured domains and 23 iterations for unstructured domains, with the number of iterations increasing by one roughly with each order of magnitude increase in problem size. It is important to recall that the HO-AMG method does not converge on a significant number of the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ test problems and, so, it is revealing that DC-all converges on all test problems.

In contrast, the BTP preconditioners require around 4–5 times more iterations to reach convergence, and the iteration counts exhibit more significant growth. With each refinement of the problem (increasing the problem size by $2\times$), an additional 2–4 iterations are needed in BTP. Despite the substantial difference in iteration counts between BTP and DC-all (Vanka), the overall solve time of BTP is only 10–30% slower for structured problems and 30–40% slower for unstructured problems. However, compared to DC-all (Vanka-DLSC), this relative solve time of BTP is 50–65% slower for structured problems and 60–70% slower for unstructured grids. This $1.5\text{--}2\times$ improvement in time to solution over DC-all (Vanka) suggests better solve phase cost-effectiveness in using DC-all (Vanka-DLSC) for $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretizations, which is supported by the relative solve phase timings per iteration. The last column of Figure 10 examines the relative total times to solution, including multigrid setup and solve phase timings. These plots are similar to the relative solve phase timings, but the values are shifted upward relative to the BTP line. This upward shift indicates that the relative setup phase for the DC solvers is longer than for BTP. As a result, DC-all (Vanka) outperforms BTP only in the unstructured case, while DC-all (Vanka-DLSC) still outperforms BTP in all cases. It is important to note that the timing results associated with the setup and solve phases are highly dependent on the implementation of relaxation setup and solution phase kernels. These timings are expected to vary depending on the specific implementation and the compute architecture.

Our results demonstrate the effectiveness of the DC-all preconditioner for solving systems discretized using the $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretizations. These findings validate the capabilities of our approach and offer valuable guidance for selecting optimal parameter values when utilizing the solver. Importantly, the comparison with the BTP

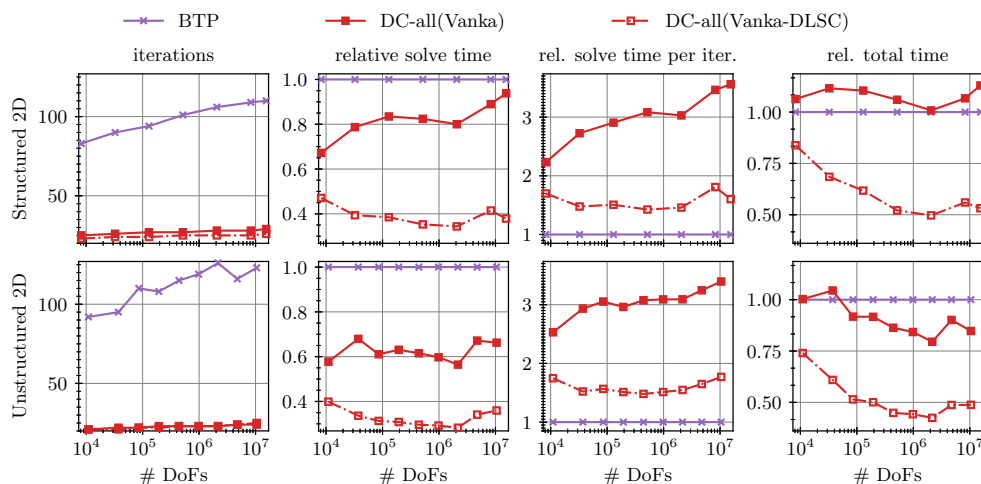


FIG. 10. $\mathbb{P}_2/\mathbb{P}_1^{disc}$ solvers: Iterations to convergence (leftmost), relative metrics compared to BTP solver-time to solution (middle left), time per iteration (middle right), and total time (rightmost), including multigrid setup and solve phase times.

method reveals the superiority of our approach in terms of iteration count and runtime, with the DC-all (Vanka-DLSC) showing promising performance improvements over Vanka-based monolithic AMG preconditioners.

5. Application problems. We conclude by examining the preconditioned FGMRES algorithms on two realistic applications pictured in Figure 11:

3D Artery: $\mathbb{P}_2/\mathbb{P}_1$. Models fluid flow inside an artery [23], discretized with unstructured tetrahedral elements. We used an STL file obtained from a luminal casting of a carotid artery bifurcation² to generate a series of meshes of increasing sizes [23]. The inflow BC is imposed normal to the bottom left cross section of the artery (see Figure 11(a), and the outflow BC (Neumann) are imposed at the opposite two cross sections, with zero Dirichlet velocities on the other boundary faces of the domain.

2D Airfoil: $\mathbb{P}_2/\mathbb{P}_1^{disc}$. This problem models flow around an airfoil [23] in the center of the domain, with an adaptively refined 2D unstructured mesh of triangular elements.³ A parabolic inflow BC is imposed along the left edge of the domain, while a homogeneous Neumann BC is imposed on the outflow boundary at the right edge, with zero Dirichlet BCs on the velocity field on all other boundary edges of the domain (including the inner airfoil).

In all tests, we adhere to the convergence criteria and data collection methods outlined in section 4. The Vanka relaxation-based AMG results reported in this section use the same solver parameters described in Tables 3 and 4. However, the DLSC relaxation-based preconditioners required enhanced approximation stages for the continuity equation and pressure correction—see lines 6 and 8 of Algorithm 3.2. For both 2D SV and 3D TH problems, we substitute Chebyshev polynomial relaxation in these stages with SA-AMG V-cycle using weighted Jacobi relaxation. This does not significantly affect the optimal damping parameters $(\omega_0^{opt}, \eta_p^{opt})$, demonstrating the parameter-free nature of the DLSC preconditioners. Additionally, for the 3D problem, increasing the V-cycle's pre- and postrelaxation sweeps from two to three iterations results in improved convergence rates.

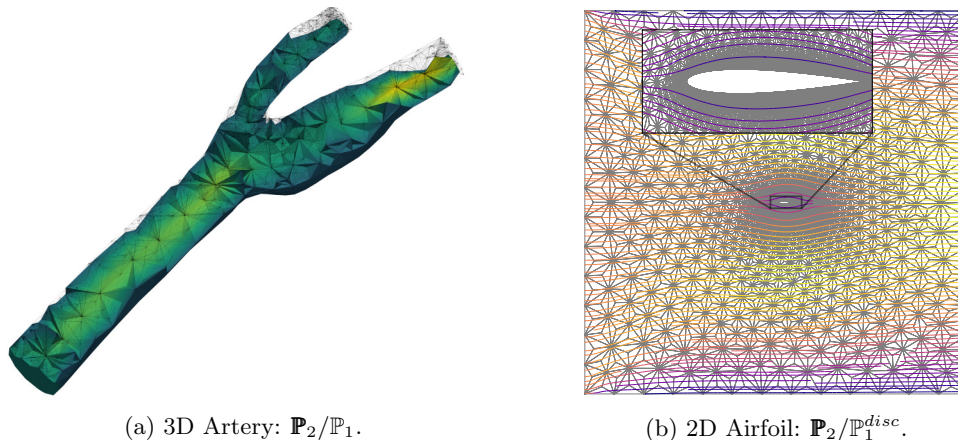


FIG. 11. Test problems for section 5.

²Carotid artery bifurcation CAD credit: <https://grabcad.com/library/carotid-bifurcation>.

³Airfoil *geo* file is available in Gmsh's GitHub repository.

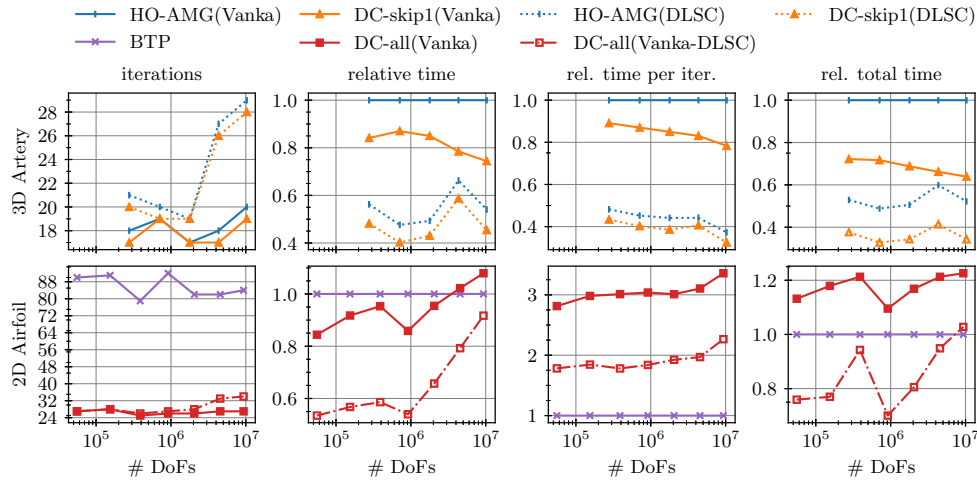


FIG. 12. Top row: 3D $\mathbb{P}_2/\mathbb{P}_1$ solvers; bottom row: 2D $\mathbb{P}_2/\mathbb{P}_1^{disc}$ solvers. Columns show iterations to convergence (leftmost) and relative metrics compared to a reference solver—time to solution (middle left), time per iteration (middle right), and total time (rightmost), including multigrid setup and solve phase times. Timings are relative to HO-AMG for 3D and BTP for 2D.

Figure 12 showcases the iteration counts for multigrid preconditioned FGMRES, accompanied by the relative time per solution and per iteration for each solver type. The top row of Figure 12 is associated with the 3D artery problem, discretized with the $\mathbb{P}_2/\mathbb{P}_1$ element pair, whereas the bottom row corresponds to the 2D airfoil problem, discretized with $\mathbb{P}_2/\mathbb{P}_1^{disc}$ elements. For the 3D problem, we compare DC-skip1 timings against HO-AMG instead of comparing with BTP. For elongated domains, the pressure mass matrix is ineffective as a preconditioner for the pressure Schur complement [28, 14], leading to poor convergence of BTP in the 3D artery problem, where BTP requires 900–1500 iterations, taking 20–30 times longer than DC-skip1. Hence, we instead compare DC-skip1 to HO-AMG.

For the 3D problem discretized with $\mathbb{P}_2/\mathbb{P}_1$, DC-skip1 solvers converge in fewer or equivalent iterations compared to HO-AMG with the same relaxation type. Thus, the relative efficiency of DC-skip1 is attributed to both reduced iteration numbers and faster individual iterations. The $\mathbb{P}_1$ iso $\mathbb{P}_2/\mathbb{P}_1$ coarse grids in DC-skip1 are sparser than $\mathbb{P}_2/\mathbb{P}_1$ grids, leading to more compact Vanka patches and decreased computational demand. Consequently, DC-skip1 solvers can handle larger problems with similar hardware resources. The performance gap is more noticeable in Vanka-based solvers due to their higher storage and computational requirements. DC-skip1 (Vanka) and DC-skip1 (DLSC) solvers are approximately 13–25% and 2–10% faster, respectively, than their HO-AMG counterparts. Despite taking more iterations to converge, DLSC-based preconditioners are faster than Vanka-based preconditioners due to the much cheaper cost per iteration, even with higher iteration counts. However, as noted in Remark 5.1, DLSC preconditioners may not be suited for all complex geometries. The relative total time column for the 3D $\mathbb{P}_2/\mathbb{P}_1$ problem follows similar trends to the relative time to solution, but with all lines shifted by 10% lower relative to HO-AMG. This indicates that even the setup for the DC-skip1 is faster due to the sparser coarse grid.

In the analysis of the 2D $\mathbb{P}_2/\mathbb{P}_1^{disc}$ airfoil problem, we conducted a comparative study between low-order DC-all preconditioners and the BTP approach. The preconditioners DC-all (Vanka) and DC-all (Vanka-DLSC) demonstrated convergence in significantly fewer iterations than BTP. In particular, for smaller-scale problems,

DC-all (Vanka) achieved solutions 5–15% more quickly than BTP. However, as the problem size increased, this advantage diminished, leading to a performance reduction of up to 8%. In contrast, DC-all (Vanka-DLSC) consistently outperformed BTP, with solution times 40–45% faster for smaller problems, but with decreased efficiency for larger problem sizes. As the problem size increases and multigrid hierarchies grow deeper, DC-all (Vanka-DLSC) becomes less competitive, and its convergence rates start diverging from those of DC-all (Vanka). A consistent trend emerges when comparing these results to those obtained from 2D unstructured meshes (see Figure 10). For smaller problem sizes, the cost per iteration of both DC-all (Vanka-DLSC) preconditioners, compared to BTP, remains relatively stable. However, a noticeable increase in cost is observed as problem sizes increase. This is attributed to denser $BM_u^{-1}B^T$ operators on coarser grids and a reduced computational efficiency of SA-AMG in preconditioning these operators. The relative total time column displays similar behavior as observed in subsection 4.2. These plots are similar to the relative solve phase timings, but the values are shifted upward relative to the BTP line due to higher setup costs. As a result, DC-all (Vanka) is always slower than BTP, while DC-all (Vanka-DLSC) outperforms BTP in all but the largest case.

Remark 5.1 (DLSC relaxation-based AMG preconditioners). In experiments not reported here, we observe that AMG solvers with DLSC relaxation often do not converge (within 200 iterations) for certain arterial simulations, a contrast to Vanka relaxation-based solvers. Attempts to enhance the DLSC solver, such as increasing the number of sweeps in DLSC stages or boosting the count of pre- and postrelaxation iterations, results in negligible improvements. The difference in performance is largely attributed to mesh quality issues, notably the presence of poorly shaped elements. To address this, we incorporated 10 iterations of Laplacian mesh smoothing during mesh generation in the present study, which improves the results. Despite this intervention, DLSC solvers still demonstrate more variability in iteration counts than Vanka solvers across problem types.

Remark 5.2 (Velocity mass matrix scaling in DLSC). The scaling of the velocity mass matrix, as outlined in Algorithm 3.2, led to reduced convergence rates for the TH discretized artery problem solvers, as depicted in the top row of Figure 12. This issue predominantly arises in the presence of challenging anisotropic meshes. In an attempt to address this, our experiments with direct inversion of the velocity mass matrix for smaller-scale, two-level hierarchies did not significantly enhance convergence rates. Remarkably, for the artery mesh, the approximation of $M_u = I$ consistently resulted in the lowest interaction counts and the fastest solution times. These findings underscore the need for a more comprehensive investigation into the implications of mass matrix scaling within the DLSC algorithm on complex meshes. This remains an important area for future research.

6. Conclusion. We present a novel algorithm for constructing AMG preconditioners for the Stokes problem using a DC approach based on the lower-order $\mathbb{P}_1$ iso $\mathbb{P}_2/\mathbb{P}_1$ discretization. By splitting the Stokes operator into block-matrix format, we simplify the assembly of algebraic interpolation operators for each variable type, thus boosting efficiency. Our approach employs the pressure stiffness matrix or an algebraic equivalent, enhancing solver robustness. This methodology can precondition both the $\mathbb{P}_2/\mathbb{P}_1$ and $\mathbb{P}_2/\mathbb{P}_1^{disc}$ discretizations. When applied directly to the $\mathbb{P}_2/\mathbb{P}_1$ discretization, this framework also results in a robust solver, albeit at a slightly higher iteration cost than low-order preconditioners. Both methods consistently demonstrate

a relatively steady iteration count across various Stokes problems on structured and unstructured meshes in two and three dimensions. Notably, the monolithic AMG preconditioners often match or even surpass the performance of inexact block triangular preconditioners across a range of problems. The robustness of the monolithic AMG approach is particularly pronounced for the elongated-domain problem, which is problematic for the inexact block triangular preconditioner.

An important advance in this work is the novel block factorization-based solution for Vanka patch systems, significantly reducing storage and computational expenses up to $2\times$ in two dimensions and $3\times$ in three dimensions. We also introduce a Chebyshev-based adaptation of the LSC-DGS [54] relaxation aimed at enhancing parallelism. This relaxation approach proves to be highly effective and efficient in well-meshed domains but less resilient in complex ones, unlike Vanka relaxation. Our AMG preconditioners with Vanka relaxation use FGMRES to automatically damp relaxation for broader applicability without intricate tuning. DLSC relaxation-based AMG preconditioners, while not needing damping parameters, do require appropriate solver and sweep count choices for each relaxation stage, which is subject to problem-dependent tuning. For the SV discretization, our findings show that optimal parameters associated with the DC framework remain stable under minor perturbations and can be efficiently determined through a coarse-parameter scan.

7. Future work. We aim to extend the $\mathbf{P}_1\text{iso}\mathbf{P}_2/\mathbf{P}_1$ -based DC preconditioners to higher-order TH and SV discretizations, by employing a p -multigrid approach. Considering the inherent challenges of such discretizations, these preconditioners are expected to be more advantageous than directly applying AMG to high-order systems. In the context of higher-order solvers, efficient Vanka implementations on fine levels are set to become an increasingly vital component of the described DC approach. With advancements in supercomputing, these higher-order Vanka methods are expected to be highly suitable for many-core systems, thanks to their expansive stencils and uniform structure. In contrast, low-order coarse-level Vanka patches, characterized by irregular dimensions and sparsity patterns, pose significant challenges in designing efficient data structures. This complexity at the coarse level underscores the utility of DLSC relaxation. DLSC relaxation, known for its efficiency and reduced memory demands, offers a practical approach to avoid the irregularities and computational hurdles associated with low-order coarse-level Vanka patches. However, it should be noted that DLSC relaxation also presents specific challenges, particularly with complex unstructured meshes, which can adversely affect convergence rates as discussed in Remark 5.2. Additionally, we plan to strengthen the theoretical foundation of our methods, aiming to derive convergence guarantees for the proposed monolithic AMG preconditioner. A key step will be ensuring the stability of coarsened velocity and pressure DoFs, essential for maintaining the discrete inf-sup condition and reinforcing our methodologies' reliability.

Reproducibility of computational results. This paper has been awarded the “SIAM Reproducibility Badge: Code and data available” as a recognition that the authors have followed reproducibility principles valued by SISC and the scientific computing community. Code and data that allow readers to reproduce the results in this paper are available at https://github.com/Alexey-Voronin/Monolithic_AMG_For_Stokes and in the supplementary materials (Monolithic_AMG_For_Stokes-main.zip [local/web 1.85MB]).

REFERENCES

- [1] J. H. ADLER, T. R. BENSON, E. C. CYR, P. E. FARRELL, S. P. MACLACHLAN, AND R. S. TUMINARO, *Monolithic multigrid methods for magnetohydrodynamics*, SIAM J. Sci. Comput., 43 (2021), pp. S70–S91, <https://doi.org/10.1137/20M1348364>.
- [2] J. H. ADLER, T. R. BENSON, AND S. P. MACLACHLAN, *Preconditioning a mass-conserving discontinuous Galerkin discretization of the Stokes equations*, Numer. Linear Algebra Appl., 24 (2017), e2047, <https://doi.org/10.1002/nla.2047>.
- [3] P.-L. BACQ, S. GOUNAND, A. NAPOV, AND Y. NOTAY, *An all-at-once algebraic multigrid method for finite element discretizations of Stokes problem*, Internat. J. Numer. Methods Fluids, 95 (2023), pp. 193–214, <https://doi.org/10.1002/flid.5145>.
- [4] P.-L. BACQ AND Y. NOTAY, *A new semialgebraic two-grid method for Oseen problems*, SIAM J. Sci. Comput., 45 (2022), pp. S226–S253, <https://epubs.siam.org/doi/pdf/10.1137/21M1429011>.
- [5] N. BELL, L. N. OLSON, AND J. SCHRODER, *PyAMG: Algebraic multigrid solvers in python*, J. Open Source Softw., 7 (2022), 4142, <https://doi.org/10.21105/joss.04142>.
- [6] M. BENZI, G. GOLUB, AND J. LIESEN, *Numerical solution of saddle point problems*, Acta Numer., 14 (2005), pp. 1–137, <https://doi.org/10.1017/S0962492904000212>.
- [7] D. BRAESS AND R. SARAZIN, *An efficient smoother for the Stokes problem*, Appl. Numer. Math., 23 (1997), pp. 3–19, [https://doi.org/10.1016/S0168-9274\(96\)00059-1](https://doi.org/10.1016/S0168-9274(96)00059-1).
- [8] A. BRANDT, *Multigrid Techniques: 1984 Guide with Applications to Fluid Dynamics*, Classics Appl. Math. 67, Springer, Philadelphia, 1984.
- [9] A. BRANDT AND N. DINAR, *Multigrid solutions to elliptic flow problems*, in Numerical Methods for Partial Differential Equations, S. Parter, ed., Academic Press, New York, 1979, pp. 53–147.
- [10] A. BRANDT AND I. YAVNEH, *Accelerated multigrid convergence and high-Reynolds recirculating flows*, SIAM J. Sci. Comput., 14 (1993), pp. 607–626, <https://doi.org/10.1137/0914039>.
- [11] J. BROWN, Y. HE, S. P. MACLACHLAN, M. MENICKELLY, AND S. WILD, *Tuning multigrid methods with robust optimization and local Fourier analysis*, SIAM J. Sci. Comput., 43 (2021), pp. A109–A138, <https://doi.org/10.1137/19M1308669>.
- [12] M. A. CASE, V. J. ERVIN, A. LINKE, AND L. G. REBHOLZ, *A connection between Scott–Vogelius and grad-div stabilized Taylor–Hood FE approximations of the Navier–Stokes equations*, SIAM J. Numer. Anal., 49 (2011), pp. 1461–1481, <https://doi.org/10.1137/100794250>.
- [13] L. CHEN, X. HU, M. WANG, AND J. XU, *A multigrid solver based on distributive smoother and residual overweighing for Oseen problems*, Numer. Math. Theory Methods Appl., 8 (2015), pp. 237–252, <https://doi.org/10.4208/nmtma.2015.w09si>.
- [14] M. DOBROWOLSKI, *On the LBB constant on stretched domains*, Math. Nachr., 254–255 (2003), pp. 64–67, <https://doi.org/10.1002/mana.200310052>.
- [15] H. ELMAN, V. E. HOWLE, J. SHADID, R. SHUTTLEWORTH, AND R. TUMINARO, *Block preconditioners based on approximate commutators*, SIAM J. Sci. Comput., 27 (2006), pp. 1651–1668, <https://doi.org/10.1137/040608817>.
- [16] H. C. ELMAN, D. J. SILVESTER, AND A. J. WATHEN, *Finite Elements and Fast Iterative Solvers: With Applications in Incompressible Fluid Dynamics*, Oxford University Press, Oxford, 2014.
- [17] M. EMAMI, *Efficient Multigrid Solvers for the Stokes Equations Using Finite Elements*, Master’s thesis, Lehrstuhl für Informatik 10 (Systemsimulation), Friedrich-Alexander-Universität Erlangen-Nürnberg, 2013.
- [18] A. ERN AND J.-L. GUERMOND, *Theory and Practice of Finite Elements*, Appl. Math. Sci. 159, Springer, New York, 2004, <https://doi.org/10.1007/978-1-4757-4355-5>.
- [19] P. E. FARRELL, Y. HE, AND S. P. MACLACHLAN, *A local Fourier analysis of additive Vanka relaxation for the Stokes equations*, Numer. Linear Algebra Appl., 28 (2021), e2306, <https://doi.org/10.1002/nla.2306>.
- [20] P. E. FARRELL, M. G. KNEPLEY, L. MITCHELL, AND F. WECHSUNG, *PCPATCH: Software for the topological construction of multigrid relaxation methods*, ACM Trans. Math. Software, 47 (2021), pp. 1–22, <https://doi.org/10.1145/3445791>.
- [21] P. E. FARRELL, L. MITCHELL, L. R. SCOTT, AND F. WECHSUNG, *A Reynolds-robust preconditioner for the Scott–Vogelius discretization of the stationary incompressible Navier–Stokes equations*, SMAI J. Comput. Math., 7 (2021), pp. 75–96, <https://doi.org/10.5802/smai-jcm.72>.
- [22] T. GEENEN, M. UR REHMAN, S. MACLACHLAN, G. SEGAL, C. VUIK, A. VAN DEN BERG, AND W. SPAKMAN, *Scalable robust solvers for unstructured FE geodynamic modeling applications: Solving the Stokes equation for models with large localized viscosity contrasts*, Geochem. Geophys. Geosyst., 10 (2009), <https://doi.org/10.1029/2009GC002526>.

- [23] C. GEUZAIN AND J.-F. REMACLE, *Gmsh: A 3-D finite element mesh generator with built-in pre-and post-processing facilities*, Internat. J. Numer. Methods Engrg., 79 (2009), pp. 1309–1331, <https://doi.org/10.1002/nme.2579>.
- [24] B. GMEINER, M. HUBER, L. JOHN, U. RÜDE, AND B. WOHLMUTH, *A quantitative performance study for Stokes solvers at the extreme scale*, J. Comput. Sci., 17 (2016), pp. 509–521, <https://doi.org/10.1016/j.jocs.2016.06.006>.
- [25] D. A. HAM, P. H. J. KELLY, L. MITCHELL, C. J. COTTER, R. C. KIRBY, K. SAGIYAMA, N. BOUZIANI, S. VORDERWUELBECKE, T. J. GREGORY, J. BETTERIDGE, D. R. SHAPERO, R. W. NIXON-HILL, C. J. WARD, P. E. FARRELL, P. D. BRUBECK, I. MARSDEN, T. H. GIBSON, M. HOMOLYA, T. SUN, A. T. T. MCRAE ET AL., *Firedrake User Manual*, 1st ed., 2023, <https://doi.org/10.25561/104839>.
- [26] Y. HE AND S. P. MACLACHLAN, *Local Fourier analysis for mixed finite-element methods for the Stokes equations*, J. Comput. Appl. Math., 357 (2019), pp. 161–183, <https://doi.org/10.1016/j.cam.2019.01.029>.
- [27] J. HEYS, T. MANTEUFFEL, S. F. MCCORMICK, AND L. OLSON, *Algebraic multigrid for higher-order finite elements*, J. Comput. Phys., 204 (2005), pp. 520–532, <https://doi.org/10.1016/j.jcp.2004.10.021>.
- [28] A. JANKA, *Smoothed aggregation multigrid for a Stokes problem*, Comput. Vis. Sci., 11 (2008), pp. 169–180, <https://doi.org/10.1007/s00791-007-0068-7>.
- [29] V. JOHN, P. KNOBLOCH, G. MATTHIES, AND L. TOBISKA, *Non-nested multi-level solvers for finite element discretisations of mixed problems*, Computing, 68 (2002), pp. 313–341, <https://doi.org/10.1007/s00607-002-1444-2>.
- [30] V. JOHN, A. LINKE, C. MERDON, M. NEILAN, AND L. G. REBHOLZ, *On the divergence constraint in mixed finite element methods for incompressible flows*, SIAM Rev., 59 (2017), pp. 492–544, <https://doi.org/10.1137/15M1047696>.
- [31] V. JOHN AND L. TOBISKA, *Numerical performance of smoothers in coupled multigrid methods for the parallel solution of the incompressible Navier-Stokes equations*, Internat. J. Numer. Methods Fluids, 33 (2000), pp. 453–473, [https://doi.org/10.1002/1097-0363\(20000630\)33:4\(453::AID-FLD15\)3.0.CO;2-0](https://doi.org/10.1002/1097-0363(20000630)33:4(453::AID-FLD15)3.0.CO;2-0).
- [32] R. C. KIRBY AND L. MITCHELL, *Solver composition across the PDE/linear algebra barrier*, SIAM J. Sci. Comput., 40 (2018), pp. C76–C98, <https://doi.org/10.1137/17M1133208>.
- [33] M. LARIN AND A. REUSKEN, *A comparative study of efficient iterative solvers for generalized Stokes equations*, Numer. Linear Algebra Appl., 15 (2008), pp. 13–34, <https://doi.org/10.1002/nla.561>.
- [34] J. LINDEN, G. LONSDALE, B. STECKEL, AND K. STÜBEN, *Multigrid for the steady-state incompressible Navier-Stokes equations: A survey*, in Proceedings of the 11th International Conference on Numerical Methods in Fluid Dynamics, Lecture Notes in Phys. 323, Springer, Berlin, 1989, pp. 57–68.
- [35] A. LINKE, L. G. REBHOLZ, AND N. E. WILSON, *On the convergence rate of grad-div stabilized Taylor–Hood to Scott–Vogelius solutions for incompressible flow problems*, J. Math. Anal. Appl., 381 (2011), pp. 612–626, <https://doi.org/10.1016/j.jmaa.2011.03.019>.
- [36] S. P. MACLACHLAN AND C. W. OOSTERLEE, *Local Fourier analysis for multigrid with overlapping smoothers applied to systems of PDEs*, Numer. Linear Algebra Appl., 18 (2011), pp. 751–774, <https://doi.org/10.1002/nla.762>.
- [37] B. METSCH, *Algebraic Multigrid (AMG) for Saddle Point Systems*, Ph.D. thesis, Universitäts- und Landesbibliothek Bonn, 2013.
- [38] A. NIESTEGGE AND K. WITSCH, *Analysis of a multigrid Stokes solver*, Appl. Math. Comput., 35 (1990), pp. 291–303, <https://www.sciencedirect.com/science/article/abs/pii/0096300390900488>.
- [39] Y. NOTAY, *A new algebraic multigrid approach for Stokes problems*, Numer. Math., 132 (2016), pp. 51–84, <https://doi.org/10.1007/s00211-015-0710-0>.
- [40] Y. NOTAY, *Algebraic multigrid for Stokes equations*, SIAM J. Sci. Comput., 39 (2017), pp. S88–S111, <https://doi.org/10.1137/16M1071419>.
- [41] Y. NOTAY, *Convergence of some iterative methods for symmetric saddle point linear systems*, SIAM J. Matrix Anal. Appl., 40 (2019), pp. 122–146, <https://doi.org/10.1137/18M1208836>.
- [42] L. OLSON, *Algebraic multigrid preconditioning of high-order spectral elements for elliptic problems on a simplicial mesh*, SIAM J. Sci. Comput., 29 (2007), pp. 2189–2209, <https://doi.org/10.1137/060663465>.
- [43] L. N. OLSON, J. SCHRODER, AND R. S. TUMINARO, *A new perspective on strength measures in algebraic multigrid*, Numer. Linear Algebra Appl., 17 (2010), pp. 713–733, <https://doi.org/10.1002/nla.669>.

- [44] A. PROKOPENKO AND R. S. TUMINARO, *An algebraic multigrid method for Q_2 - Q_1 mixed discretizations of the Navier-Stokes equations*, Numer. Linear Algebra Appl., 24 (2017), e2109, <https://onlinelibrary.wiley.com/doi/pdf/10.1002/nla.2109>.
- [45] J. QIN, *On the Convergence of Some Low Order Mixed Finite Elements for Incompressible Fluids*, Ph.D. thesis, The Pennsylvania State University, 1994.
- [46] J. SCHÖBERL AND W. ZULEHNER, *On Schwarz-type smoothers for saddle point problems*, Numer. Math., 95 (2003), pp. 377–399, <https://doi.org/10.1007/s00211-002-0448-3>.
- [47] M. UR REHMAN, T. GEENEN, C. VUIK, G. SEGAL, AND S. MACLACHLAN, *On iterative methods for the incompressible Stokes problem*, Internat. J. Numer. Methods Fluids, 65 (2011), pp. 1180–1200, <https://doi.org/10.1002/fld.2235>.
- [48] S. P. VANKA, *Block-implicit multigrid solution of Navier-Stokes equations in primitive variables*, J. Comput. Phys., 65 (1986), pp. 138–158, [https://doi.org/10.1016/0021-9991\(86\)90008-2](https://doi.org/10.1016/0021-9991(86)90008-2).
- [49] P. VANĚK, J. MANDEL, AND M. BREZINA, *Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems*, Computing, 56 (1996), pp. 179–196, <https://doi.org/10.1007/BF02238511>.
- [50] A. VORONIN, Y. HE, S. MACLACHLAN, L. N. OLSON, AND R. TUMINARO, *Low-order preconditioning of the Stokes equations*, Numer. Linear Algebra Appl., 29 (2022), e2426, <https://onlinelibrary.wiley.com/doi/abs/10.1002/nla.2426>.
- [51] A. VORONIN, R. TUMINARO, L. N. OLSON, AND S. MACLACHLAN, *Algebraic Multigrid Based on Low-Order Systems*, Technical report, 2021, <https://www.sandia.gov/app/uploads/sites/210/2022/05/CSRI-2021-proceedings1.pdf#page=155>.
- [52] M. WABRO, *Coupled algebraic multigrid methods for the Oseen problem*, Comput. Vis. Sci., 7 (2004), pp. 141–151, <https://doi.org/10.1007/s00791-004-0138-z>.
- [53] M. WABRO, *AMGe—Coarsening strategies and application to the Oseen equations*, SIAM J. Sci. Comput., 27 (2006), pp. 2077–2097, <https://doi.org/10.1137/040610350>.
- [54] M. WANG AND L. CHEN, *Multigrid methods for the Stokes equations using distributive Gauss–Seidel relaxations based on the least squares commutator*, J. Sci. Comput., 56 (2013), pp. 409–431, <https://doi.org/10.1007/s10915-013-9684-1>.
- [55] R. WEBSTER, *Stability of algebraic multigrid for Stokes problems*, Internat. J. Numer. Methods Fluids, 71 (2013), pp. 488–505, <https://doi.org/10.1002/fld.3672>.
- [56] S. ZHANG, *A new family of stable mixed finite elements for the 3D Stokes equations*, Math. Comp., 74 (2005), pp. 543–554, <https://doi.org/10.1090/S0025-5718-04-01711-9>.
- [57] S. ZHANG, *On the P_1 Powell–Sabin divergence-free finite element for the Stokes equations*, J. Comput. Math., 26 (2008), pp. 456–470, <http://www.jstor.org/stable/43693550>.
- [58] S. ZHANG, *Quadratic divergence-free finite elements on Powell–Sabin tetrahedral grids*, Calcolo, 48 (2011), pp. 211–244, <https://doi.org/10.1007/s10092-010-0035-4>.